A Comparison of Software Architectures through EEG-based emotional classification using Machine Learning

By

Cian Connolly

A thesis submitted to Macquarie University
in partial fulfilment of the degree of Bachelor of Engineering with Honours
School of Engineering (Software)
November 2025
Thesis Supervisor: Subhas Mukhopadhyay



MACQUARIE
University
SYDNEY·AUSTRALIA

Contents

Contents iii

Abstract v

Statement of Originality vii

Acknowledgements ix

1 Introduction 1
- 1.1 Overview . . . 1
- 1.2 Thesis Scope and Research Questions . . . 2
  - 1.2.1 Research Questions . . . 2
  - 1.2.2 Scope . . . 3

2 Literature Review 5
- 2.1 Software Architecture . . . 5
  - 2.1.1 Layered Architecture . . . 6
  - 2.1.2 Pipes and Filters . . . 7
  - 2.1.3 Microkernel . . . 8
- 2.2 Electroencephalography . . . 9
  - 2.2.1 Pre-Processing . . . 9
- 2.3 Emotional Classification . . . 11
  - 2.3.1 Categorical Model . . . 11
  - 2.3.2 Dimensional Model . . . 12
- 2.4 Artificial Intelligence learning approaches . . . 13
  - 2.4.1 Classical Machine Learning . . . 13
  - 2.4.2 Deep Learning Models . . . 14

3 Methodology 17
- 3.1 Hardware Setup and Usage . . . 18
- 3.2 Software Tools . . . 19
  - 3.2.1 Workflow Software . . . 21
- 3.3 Prototype Pipeline Development . . . 21
  - 3.3.1 Dataloader . . . 22
  - 3.3.2 Pre-processor . . . 22
  - 3.3.3 Segmentation . . . 23
  - 3.3.4 Feature Extraction . . . 24
  - 3.3.5 Model Creation and Evaluation . . . 25
- 3.4 Architectures . . . 25

3.4.1 Layered Architecture . . . 25
3.4.2 Pipes and Filters . . . 27
3.4.3 Microkernel . . . 28
3.4.4 Dataflow Diagram of Architecture Testing . . . 31
3.5 Live Inference . . . 31
3.5.1 Initial Testing . . . 32
3.5.2 Developing the Dataset and Script . . . 32
3.5.3 Dataflow Diagram of Live Inference Setup . . . 32

4 Preliminary results **35**
4.1 Overview and Preface . . . 35
4.2 Comparing Frequency Bands . . . 35
4.3 Visualising Data Artifacts . . . 43

5 Results & Discussion **45**
5.1 Machine Specifications . . . 45
5.2 Architecture Performance . . . 45
5.2.1 Runtime Duration . . . 46
5.2.2 Memory Usage . . . 47
5.2.3 CPU Utilisation . . . 48
5.3 Model Performance . . . 49
5.3.1 Architectures Relationship to Model Performance . . . 49
5.4 Live Inference . . . 52

6 Conclusion & Future Work **55**
6.1 Conclusion . . . 55
6.2 Future Work . . . 55

7 Appendix **57**
.1 Github Repository . . . 57
.2 Custom Song Dataset . . . 57

A Ethics **59**

References **61**

Abstract

Artificial intelligence (AI) systems are being rapidly created and deployed with a disproportionate amount of research into how best to realistically develop efficient and optimal software solutions. These new problems require well-informed decisions, and currently, information to assist in making those decisions is inadequate. AI has unlocked new possibilities in how we interact with electroencephalography (EEG), allowing us to develop new kinds of systems that allow programs to interface directly with our brain waves. This has positive repercussions in medical fields, detecting neurological conditions as well as massively furthering accessibility possibilities with brain-computer interfaces, allowing people with physical disabilities to gain a new avenue of interfacing with the world. Using EEG and AI together opens potential for all sorts of new interactions, for example, controlling mechatronic systems with our brains or understanding someone's emotional state under certain stimuli.

This project is looking to advance these EEG to AI pipelines by exploring different software architecture solutions and measuring their performance against each other. Additionally, the project also explores whether our brains do, in fact, act differently under different stimuli and whether that is a measurable process. It looks at whether programs can be developed to infer on live EEG data and make labelled predictions on whether a person is listening to more pleasant or unpleasant songs. These questions were explored through the creation of software designed in different architectures, designed to take in EEG data from an available database, such as DEAP or through at-home solutions like a Muse 2 headset, and pre-process this data effectively for machine learning. Multiple different models were then created with their metrics analysed to find the best performing model. The best performing model was then built into a live inference software solution that performed successful live inference on a custom-made single-person dataset.

The architectures built and discussed were the Layered, Pipes-and-Filters, Microkernal, and an original software prototype that is unbound to any structure. The Layered and Pipes-and-Filters both performed substantially better than microkernal, which showed to be significantly less efficient than the aforementioned. The Layered architecture also showed more sporadic results, where the Pipes-and-filters succeeded in consistency. The best machine learning model tested was a Random Forest model utilising 500 tree's which reached a height of 90% in its F1-score.

This research can help provide insight into software projects when evaluating their Architecturally Significant Requirements (ASRs), better informing teams to make decisions that will prevent problems arising down the line of software development. Architectural problems that arise in later software development can lead to large additional unplanned costs, wasted time, and inefficient code. By making better decisions, programmers can lower development times and costs of software development.

Statement of Originality

This work has not previously been submitted for a degree or diploma in any university. To the best of my knowledge and belief, the thesis contains no material previously published or written by another person except where due reference is made in the thesis itself.

Any contribution by other people, including co-authors of papers, has been clearly stated in the Authorship Contribution Statements.



Cian Connolly

4/11/25

Date

Statement of Using Generative AI and Editing Tools

I, Cian Connolly, acknowledge the use of artificial intelligence (AI) tools in the research project and the writing of this thesis.

In this work:

- I used Grammarly (Free) [53] to help me with spell checking and grammar.
- I used ChatGPT V3.5 and V5 [56] to help me with identifying gaps in background areas unfamiliar to my software education, namely in neuroscience and EEG related areas.
- I used ChatGPT V3.5 and V5 [56] to help me with troubleshooting errors in LaTeX and in code.
- I used Macquarie University Library's Primo Research Assistant [57] to help me with finding relevant literature

I am aware of the AI tools limitations and ultimately, I am responsible for the content, research and developing my ideas described in this document.

Signature: 
Date: 5/11/25

Acknowledgements

This thesis would not be possible without the support of a few very important people. First and foremost has been the amazing Subhas Mukhopadhyay, Professor of Mechanical/Electronics Engineering at Macquarie University, who has overseen this project and provided invaluable time and attention to helping get the thesis to the stage it is at today.
Next, and definitely not least, has been Kate Stefanov, who has been critical in guiding the process of how to research, write, and present this thesis. This work would not be nearly as put together without her assistance.

Music is the shorthand of emotion
Leo Tolstoy

1 Introduction

1.1 Overview

Artificial Intelligence (AI) is a rapidly developing field in all areas of technology, and as such, the structure in which AI programs are designed and maintained has lagged behind in research. With the sheer multitude of new AI programs being developed every day, programmers are making uninformed or outdated decisions on which software architecture to use as the scaffold for their solutions. As such, there is meaningful research to be done on how these different architectures perform to help inform new programmers early on with how to build their software bases.

One such area of concern where AI has been largely beneficial is in the health sector. Medical professionals have already been able to incorporate different machine learning models to assist in diagnosis and response time in their field. Machine learning has been proven to exceed doctors in image-based diagnoses in several areas, such as radiology, dermatology, pathology, and cardiology [1], all physical medical fields; however, AI has stayed behind in assisting mental-based medical fields such as mental health and learning disabilities. This project highlights one such field in this mental health environment, a cognitive-AI interface using electroencephalography (EEG). There are various potential avenues where understanding how the brain is feeling can lead to individuals' benefits, for example, we can gain a better understanding of depression, improving early detection and treatment. We can also improve meditation and self-help by creating significantly more individualised programs using real-time adjustments based on the user's emotional reactions. Further, we can provide assistive technology for speech or movement impairments.

This project looks through the lens of EEG, and importantly, how the emotional stimuli of music can affect the brain. Music has been at the forefront of emotional conveyance through human history, and as such is a known method of evoking emotional responses in people. This project starts with the initial question that although we know presumptuously that music can lead to a change in our emotional states, is this change tangibly measurable through technology such as EEG? This question is well researched, and a large portion of the groundwork of data collection has been laid out by the academic community, as they have fueled this research through the production of different EEG datasets that all record

participants listening to music.

One problem that arises from these datasets, however, is cross-subject and cross-device variability. As our brains are all different, and these datasets often are comprised of a small number of people in relation to the human population, the models trained on these sets can often lead to high accuracy for the people in the datasets, yet perform poorly on those outside the dataset. Additionally, different EEG collection devices collect brain activity using electrodes placed in different locations, and as such, prevent separate devices from using models built on other devices, as the electrode placements must match. This project aims to test whether a dataset built on one person using the same device can be used to infer live emotional states of the individual, an idea that looks to the opposite end of cross-subject variability to determine if perhaps it may be more reliable to specialise individual models for inference.

1.2 Thesis Scope and Research Questions

1.2.1 Research Questions

This project's goal is to explore and measure positive, neutral, or negative emotional reactions to different songs. This will be done through the data collection, training, tuning and experimenting of different machine-learning artificial models. Additionally, measures to mitigate cross-subject variability will be implemented, and their effectiveness evaluated and measured. Three research questions are being investigated:

1. R1) Can we test and compare the performance of different software architectures in handling the performance of an EEG emotional-classification pipeline?

2. R2) Can we identify the best machine learning model for EEG-based emotional recognition

3. R3) How does a person's emotional state react to different emotional stimuli in a music medium? Can we accurately measure positive, negative or neutral changes?

4. R4) Can we create a program that performs live inference on a person's emotional state?

R1 is the priority of this project and aims to look at developing programs in different software architectures to provide insight into performance metrics to help inform decision-making when choosing architectures for similar projects.

R2 will also look to contribute to the performance of specific algorithms in EEG-based emotional recognition and to test whether results are impacted at all by the performance of different software architectures.

R3 will be discussed in the literature review, with further discussion of the collected results from this project's experiments.

Lastly, R4 will test whether the roadblocks found in academia around cross-subject variability and cross-device variability can be replicated and then mitigated through experiments on a single subject with a single device. This will be done through live inference of the best trained model from the project.

1.2.2 Scope

This project looks to develop a program that compares the resource performance of different software architectures: pipes and filters, layered, microkernel, and no-architecture. Within these programs, different machine learning models will be built and tested to evaluate the performance of different AI models in EEG-based emotional recognition. The models within the scope of this project are traditional machine learning models, without the testing of more dimensionally complex deep learning. Due to time and resource restrictions, these complex models could not be integrated into the project.

The program will work to prepare data, pre-process the data, segment the data and perform feature extraction on the segments. The entire pipeline will be inside the program.

The metrics for the performance of software architectures are time, CPU usage, and memory. Whereas, metrics for the performance of the AI models are accuracy, precision, recall, and F1-score.

A script to then take the best-trained model and use it for live inference will be developed to test the feasibility of inferring on live EEG data rather than evaluating performance metrics. Performance metrics such as accuracy and lag are not looked at in this paper.

2 Literature Review

In this chapter review, I'll provide the background research already present within the field as well as important relevant research that has been conducted adjacent to the project. The project explores the efficiency of different software architectures when handling machine learning, as well as the performance of various algorithms when inferring on electroencephalographic (EEG) data.

The first section of the literature review will look into the software question of discussing possible architectures for the program. Whilst the second section will explore how to process EEG data by outlining how we can collect and use EEG data to provide inputs for a system. Additionally, outputs will need to be categorised into tangible labels for machine learning, and as such, emotional classification is touched on, and lastly, an analysis of artificial intelligence in the field also starts to provide a base understanding and clear comparisons that will be required for our own model development, as well as choosing models to test. The term "Artificial Intelligence" is vague and can be further clarified as classical Machine Learning algorithms and Deep Learning architectures for this paper.

2.1 Software Architecture

Software Architecture is a crucial cornerstone of software engineering and development. Within the industry, there is an overall consensus on the important role good software architecture holds in dictating important factors of code development[2]. These factors often focus on shortening the software development cycle, which in turn frequently leads to lower development costs[3].

As machine learning grows more powerful, accessible, and useful, software architecture's relationship with machine learning pipelines needs to be further researched. These pipelines often have a large focus on performance and scalability[4], and in these earlier stages of rapid artificial intelligence development, there is also a large focus on evolving requirements, where often software engineering teams do not have the experience to plan a full system architecture[5] and even less experience in maintaining these architectures[6] [7]. This further increases the relevance of this research to better understand how different known

architectures can act as a strong baseline for new machine learning development.

Architecture is often decided early in development, as shifting across different options can lead to greater overhead and wasted time. As such, it is important to understand the strengths and weaknesses of different architectures to allow for informed decision-making early on. Due to this major decision happening early on, problems can arise, often in various software systems, and the planned architecture is changed throughout development[8]. The later changes to a planned architecture are made, the more costly they are to pivot to a new architecture, and as such, confidence in an architecture is highly relevant when selecting one[8].

This paper aims to provide insight into the performance of different architectures and evaluate them through distinct metrics to conclude how they perform on handling a machine learning pipeline and therefore help teams in early planning to make further educated decisions in architecture selection. To determine what architectures to test, it is strongly recommended to determine a system's Architecturally Significant Requirements (ASRs), which are aimed at providing insight to determine an optimal architecture. The key part of an ASR is the definition of significant, with Bass et al. stating that an ASR is any requirement where omitting it would lead to a profound effect on the architecture, where, without the requirement, the entire architecture would be different[9]. Whereas Chen et al state their definition of "significant" as any change that would have a high cost associated with it[10]. Together, these definitions ultimately paint a picture where the crucial factors to consider first and foremost when determining a software architecture are what the primary outcomes of the project are, and, throughout the development of the program, what are the predictable changes that may lead to high costs, likely through an increased development time.

This project's ASRs lead to the investigation of three different plausible architectures:

- Layered Architecture
- Pipes and Filters Architecture
- Microkernel Architecture

2.1.1 Layered Architecture

Layered Architectures are the most common in industry; they naturally lend themselves to larger teams due to their structure, which focuses on breaking down a system into a series of hierarchical layers. These layers have clear rules of abstraction among other layers in the system, and often encapsulate a full subtask[11]. Due to its commonality in industry, the architecture acts as a good control for the project, whilst also being a relevant and powerful architecture.

The layered architecture excels in larger teams that have broken off into smaller subtask teams. Due to the abstraction, teams can often focus on the integration of their layer without needing too much information on other layers. For deep learning, this would manifest through the separation of subtasks often present when processing datasets and running models on them. There is a clear linear pathway for the data to pass through: a dataloader, preprocessor, segmentation, feature extraction, model creation, and model inference. This pathway does subscribe to the layered structure intuitively, while also mitigating one issue with the layered architecture, the sometimes unpredictable amount of layers that are required for the system as a whole. Frank Buschmann et al. have criticized the layered architecture for this issue, knowing how many layers is essential for the structure to function correctly as they raise

how issues come from having to many layers or too little[12][13], however, our pipeline of machine learning has clear and well-known tasks and as such, this issue is only relevant when new changes to the pipeline are wanting to be made.

A layered architecture does come with drawbacks, especially when compared to other, more flexible architectures. Often, a layered architecture is more inefficient and can lead to more redundant work being done. Abstraction between subtasks can lead to duplicate work, and the hierarchical nature of the structure leads to major changes sometimes being made when one other layer goes through a drastic change or re-planning[11]. There are also concerns with fault tolerance, specifically that a fault in any layer can sometimes lead to a system crash[14], as the lower layers require the higher layers to finish their task successfully before they can start. Further issues also arrive when projects are scaled; layered architectures are more difficult to scale than other layers due to their stricter guidelines for data flow between layers[15].

In the context of a simple pipeline for machine learning, a layered architecture can often meet the requirements of ASRs, the linear pathway of machine learning attributes itself well to the structure, and the architecture will likely work in most cases; however, the weaknesses of the architecture can often outweigh this convenience, and the extra work time, decreased efficiency, and scalability in production time can be solved with more flexible architectures, Nazir et al. emphasizes a major challenge in machine learning pipelines being the need to be flexible and scalable[6], thus although a layered architecture may produce a functional system, it likely will fall short in the long term development of machine learning.

2.1.2 Pipes and Filters

The major difference that isolates a Pipes and Filters architecture from a more traditional layered architecture is how each structure handles the data flow of the system. In a layered architecture, data is controlled through hierarchical layers, where higher layers rely on lower layers, and then perform a specific function of the system. Whereas in a pipes and filters structure, data flows through the system, with each filter performing its task before passing results to the next filter as soon as they are ready[12]. These filters are often smaller tasks than a layer, as they aren't looking to perform a full function of a system, but instead a part of that function that modifies the data[9]. Filters can be thought of as independent components that, combined, compose the entirety of a system. These components need some form of pushing or pulling the data through this system, and these connecting channels are known as pipes.[16].

A major characteristic of the pipes and filters architecture lies in its strengths of modularity; a filter only requires the truth of its input and output, and as such, the internal workings of a component are freer of structure[11]. This allows the structure to easily accommodate the replacement of filters and, further, with the introduction of new pipes, allows for the inclusion of new filters into the system's pipeline[17]. This strength lends itself to the program developed for this project, as modularity allows for more thorough testing of different machine learning pipeline configurations, leading to easier integration of alternative setups in prepossessing and feature extraction. Expanding out from this, we see a supporting use case for pipes and filters in larger machine learning systems, where modularity and flexible development lead to simpler alterations to any part of the machine learning pipeline, a multifaceted stream of operations that has many areas where changes can be made to further the effectiveness of the entire system.

Importantly, however, are the criticisms that come from the simplicity of pipes and filters

modularity. Bass et al. dive into some issues that present themselves with component-and-connector (C&C) architectures, like pipes and filters[9]. Some issues that sometimes arise with modular architectures are increased overheads, redundant code, a lack of re-usability where possible, and unnecessary complexity. Often, these problems can be resolved with accurate planning, yet they are important to note.

A problem more specific to pipes and filters and very reliant for machine learning data flows is the importance of filters having a common data format. Scheibler et al. speak on this idea and how this problem can lead to additional work for every filter that wishes to be integrated into the system, which leads to reduced performance, a primary metric for machine learning [17]. However, when it comes to machine learning dataset preparation, the actual format of the data doesn't change too frequently; it's often a series of mathematical formulas applied to a dataset, often a tensor, and as such, the format doesn't require many changes through the pipeline's functions, mitigating this issue drastically. Scheibler et al. further comment on another impact on performance for pipes and filters, each pipe in the system requires resources to be allocated leading to larger scale pipes and filters systems with many pipes to lose a significant amount of efficiency in just moving the data around, Scheibler et al. argue that pipes and filters work better for smaller systems then larger ones because of this, Bass et al. also cautions about this weakness for the architecture suggesting C&C structures function better when smaller.

2.1.3 Microkernel

The microkernel software architecture is another structure that looks to prioritise modularity. Conceptually, a microkernel program has two major parts: a core system (often known as a kernel) that implements the minimal mechanisms to keep the system functional, and the second part, a series of plugins (also known as user-space modules) that, when connected to the core system, each add their own new capabilities to improve the functionality of the system [15].

The architecture gained popularity as operating systems grew larger, and their monolithic structure led to problems with complexity; the more functionality added to an operating system, the larger the kernel would need to expand. This led to the demand for a more modular and flexible approach that also had significantly stronger security capabilities. Thus, the microkernel grew in popularity, especially around operating system development [18].

However, this doesn't mean that the architecture is restricted to operating system problems. In his paper, Vick Dini explored the use of a microkernel in web-based architectures and found it plausible in managing some of the challenges faced with either monolithic or microservice architectures[19]. The paper looks at how the microkernel's nature of being able to manage a system's resources, whilst having plugins handling login flow, works outside of operating systems; importantly, for an EEG pipeline, these are two key areas of requirements. Management of resources is crucial in all things machine learning due to the large dataset sizes matched with highly computational processes, and as such, this makes the architecture a great candidate for analysis in an EEG pipeline where both modularity and performance are being investigated.

A microkernel's modularity comes from how new functionality can be implemented into the system through the creation of a plugin. These plugins can range in granularity from the smaller functions similar to filters in a pipes and filters architecture, or larger whole subsystem functions like a layer in the layered architecture. In reality, however, the plugins often encapsulate independent components that house their own logic flow[15] and,

as such, for this project will more likely hold similarities to layers. The big difference between microkernel and layered architecture though is the control of the dataflow. A layered architecture looks to restrict process communications between adjacent layers, whereas for a microkernel, the control is given to the kernel through communication protocols, and the plugins don't communicate with each other. This is a large part of what makes the microkernel more modular than the monolithic layered approach, the flexibility to remove and add in functionality free from communication ordering restraints. Yet, it does add increased overheads on the kernels performance.

Where the microkernel architecture likely struggles for this program is in efficiency. A known weakness of the architecture is in performance issues, largely due to the communication overheads mentioned. The increased amount of messages being passed between plugins and the kernel leads to frequent context switching to determine which plugin is needed. Microkernel architectures traditionally did not strive for high performance, as pointed out by Chen et al. [20] who looked to improve the performance of the architecture for emerging technologies. They found that although SOTA microkernels like seL4 [21] have achieved much higher Instructions Per Cycle (IPCs) than other microkernels, these highs still include significant performance issues in certain cases. Additionally, performance problems were also found with memory footprints and other operating system-related metrics.

For this project, as we will be primarily comparing performance metrics of the different architectures, it is expected that the microkernel will perform the worst. However, a comparison with the performance in a machine learning pipeline will be unique, and if performance woes are minuscule compared to the potential benefits of the architecture, there may be use cases for the architecture in the field.

2.2 Electroencephalography

Electroencephalography (EEG) measures electrical activity in the brain through the placement of electrodes around one's head, placed in specific areas on the scalp, where the electrodes detect changes in voltage. This change in voltage can be extracted in a numerical form that can then be used as inputs for an AI model. EEG data is collected through different configurations of electrodes, commonly either bipolar or referential montages. A bipolar montage has each electrode's represented voltage as the difference between its two adjacent electrodes, whereas in a referential montage, the electrode's signal is calculated based on the voltage difference between the electrode and the average of all other electrodes[22]. This understanding of how EEGs collect data is essential for understanding the pipeline of data in the project. There are different ways to treat this data, and as such, the way it is collected is important. Common issues around these different configurations arise when datasets have either different configurations or different devices. There needs to be a uniform standard for the input data; these changes being present in a dataset can lead to errors or bad data.

2.2.1 Pre-Processing

Raw EEG data can be pre-processed in multiple ways using different transformation algorithms. A study by Souvik Phadikar [23] looked at the advantages and disadvantages of these different methods, as well as their overall effectiveness. He concluded that algorithms that perform on the frequency-domain had a higher performance than those on the time-domain.

Fourier Transform

The Fourier Transform is an algorithm that converts signals collected in the time-domain into signals in the frequency-domain. As EEG collects voltage changes over a period of time, the data starts in the time-domain and requires this change if we want to do effective analysis[23][24]. The common formula for this Fourier Transform is the Discrete Fourier Transform (DFT) defined as:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi \frac{k}{N} n} \tag{2.1}$$

The DFT algorithm is inefficient, running at a time complexity of $O(N^2)$; however, there is a faster alternative. The Fast Fourier Transform (FFT) brings this complexity down to $O(NlogN)$ by applying divide-and-conquer techniques to break down the data before applying DFT.
By doing an FFT pass on the raw data we are given a new output of frequency bands. The five different frequency bands can be related to different mental states, and in this project will be used to assist in making sure subjects are in a similar state before being shown emotionally charged stimuli[25][26].
The states are as follows:

Table 2.1: List of Frequency bands and their associated brain states

Frequency Band	Frequency Range (Hz)	Associated Brain State when more prevalent
Delta (δ)	0.5–4	Sleepiness
Theta (θ)	4–8	Deeply relaxed, emotionally stable
Alpha (α)	8–12	Relaxed, passive attention
Beta (β)	12–35	Anxiety dominant, active, external attention
Gamma (γ)	over 35	Infomation Processing

Wavelet Transformation

Another common pre-processing technique is Discrete Wavelet Transformation (DWT). Unlike FFT, DWT works on the time-domain data of the EEG and decomposes the signal data into specified frequency bands. The algorithm works very well for more dynamic EEG data, such as detecting dramatic shifts of emotions which in academia is very necessary for studying emotional classification. However, Phadikar et al.'s research found that although promising, the need to find a "mother wave" in DWT makes it significantly more limiting than FFT and other algorithms[23].
More studies have further explored Wavelet Transformation in machine learning models for emotional classification, with the most significant advancement being the development of the Flexible Analytic Wavelet Transformation (FAWT). FAWT has been able to create significantly advanced models with one study by Vipin Gupta developing a Random Forest model that achieved an accuracy of 90.48% on classifying positive, negative and neutral emotions[27] on the SEED dataset[28].
FAWT aims to address the primary issue with DWT and other Wavelet Transformation by preventing the mother wave from being a rigid decision that may lead to poor results, and instead allow it to be significantly more adaptive [29].
The primary issue with FAWT is the difficulty of implementation when compared to a Fast Fourier Transform. It requires a substantial amount of knowledge of EEG signals and signal

processing, and is still fairly new so there are not many open-source tools to assist in applying it. Due to this, this study will use FFT as it is implemented already in various tools such as Mind Monitor and Matlab. For future studies, changing this pre-processing could result in higher accuracy in models.

2.3 Emotional Classification

With this understanding of how an AI model can use EEG data as input to a machine learning model, the models' outputs must also be defined as tangible categories or labels. To create a model that outputs what emotion is associated with the input EEG, there must be a method of categorising emotions. Multiple models have been proposed in academia for decades, and there is no strongly agreed-upon model for classifying emotion.

2.3.1 Categorical Model

There is a lot of research on the thought process of categorising emotional states as discrete emotions. In 1992, Paul Ekman proposed the "basic emotion model" which proposed that emotions could be discretely categorised as "6 basic emotions"[30]. These six emotions were joy, anger, disgust, surprise, sadness and fear; Ekman argued that any person could only feel one of these emotions at one time. Issues arise with this model's broadness and its inability to represent differing levels of one emotion, for example, the happiness one feels when getting married is often significantly more potent than the joy of listening to a happy song. Additionally, this model disregards the possibility of "mixed emotions", a term starting to pop up more frequently in emotional categorical discussions[31]. An example from Christina Ting Fong was her finding that mixed emotions have been felt by students on graduation day, where there are both feelings of happiness and sadness at the same time[32] – a direct contradiction to Ekman's proposition.

Over time, the basic emotion model has been further developed to account for these other emotional feelings, and is now more colloquially known as the categorical model. A major issue that the categorical model has for this project is the lack of consistency in academia. One of the most common categorical models is Plutchik's wheel of emotions as seen in figure 2.1, which further adds trust and anticipation to Ekman's six basic emotions. The model also starts to address the concerns with Ekmans' model by introducing more variability of the potency of an emotion, and by introducing more complex emotions that exist as a combination of two other emotions. Other adjustments to Ekman's models include various other research papers introducing their own emotions to Ekman's, often further classifications for what is relevant for their study, such as "news-related" for a social media based study[34], or "criticism", "sarcasm" and again "news-related" for a Biomedical classification[35].

The categorization model is heavily limited due to its inconsistency across the field; a need for discrete classifications leads to arbitrary amendments to existing models such as Ekman's or Plutchik's. This project's primary goal is to increase the subject generalization of a machine learning emotional classification model, and as the categorisation model often relies on context-specific criteria based on a study's goals, it is too situation-specific to be utilized for this project.

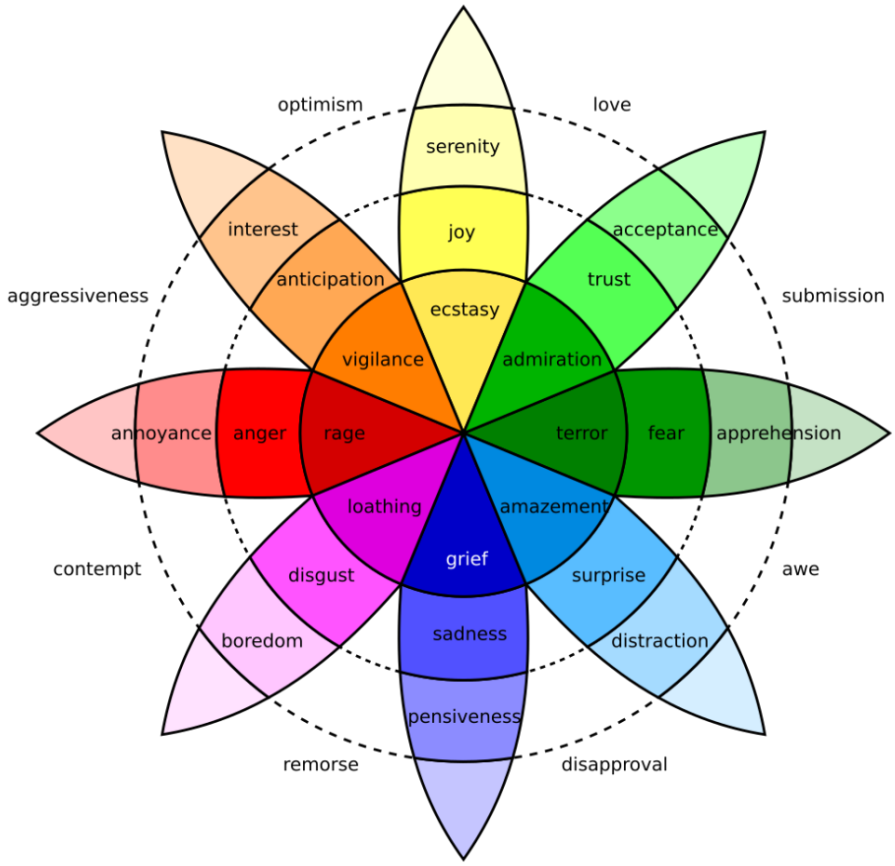


Figure 2.1: Plutchik's Wheel of Emotion[33]

2.3.2 Dimensional Model

The second major model utilised in emotional classification is the dimensional model, whereas instead of classifying emotions as discrete entities, they can be graphed across a constant circumplex. There have been various proposed circumplex models to debate how best to define the axis of the graph with some introducing a third dimension to the model[36].

In academia, the most common model agreed upon is Russell's arousal against valence model[38] seen in figure 2.2. The model works as a scale across a range of emotions, allowing the representation of many more feelings. Rather than categorising a feeling as happy, it may be graphed on the model in such a way that shows they are happy, but also energetic, thus putting that person closer to excited. There is a significant amount of flexibility in this model and it comes as a strength of the model, but also a detriment.
The model suffers from a lack of clarity, an emotion becomes a point on a graph that could be inferred differently by multiple people. A point closer to the middle starts to represent a neutral state, yet at what point is that neutral state closer to resembling an emotion? At what point does miserable stop being miserable and instead become neutral? This issue extends further into cross-subject variability, people may feel certain emotions more strongly or weakly than others, and the location of emotions on the graph would vary from person to person.

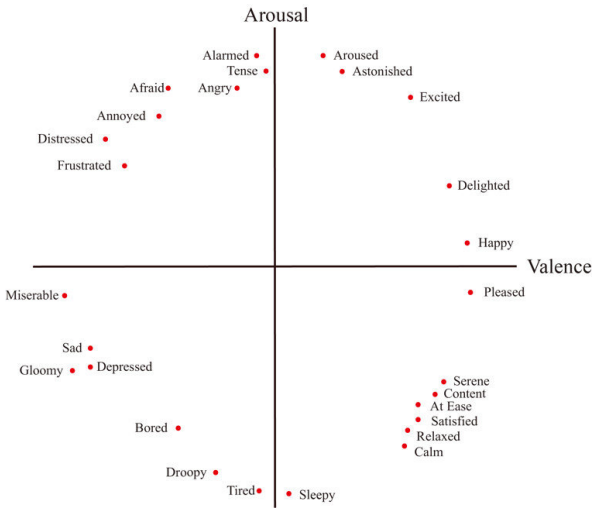


Figure 2.2: Russel's Dimensional Model[37]

Adjusting the Dimensional Model

Although utilising the dimensional model for distinguishing a specific emotion someone may be feeling, the measurement of valence and arousal can still be used effectively for the project. A simplified version of the model has been used in other studies to measure broader emotions such as happiness and sadness (positive or negative valence)[39][40] while focusing on arousal levels to determine a relaxed state. This broader alternative allows us to build models that can evaluate if someone's current emotional state is positive, negative or neutral, lessening the categories required and still allowing us to test accurate emotional shifts in subjects.

2.4 Artificial Intelligence learning approaches

Artificial Intelligence algorithms can be split into two different subheadings: Classical Machine Learning and Deep Learning. Classical Machine Learning is more explicit with the rules and mathematical functions that dictate them, whereas Deep Learning often utilises neural networks to incorporate layers of neurons to self-train and find patterns within the input data.

2.4.1 Classical Machine Learning

Classical Machine Learning often hovers between 50% - 90% accuracy in EEG-based studies in predicting emotional states. A study by Hassan Khalid S. Alsayigh1 and AbdulSattar M. Khidhir used multiple algorithms to test a subject's attention levels; random forest classification was found to be the most accurate with a 91.40%. However, they also had commendable results using a Support Vector Machine (SVM) at 82.70% accuracy, a logic regression model at 83% accuracy, and a Decision tree with 81% accuracy. They also had more inaccurate models that did not achieve great results, notably a Gaussian Naive Bayes algorithm that achieved a 60% accuracy and a K-Nearest Neighbors(Knn) algorithm with a 77% accuracy[41].

The Random Forest Classifier here provides the best model, and this is supported as well

by a study by Vipin Gupta et al.[27]. Gupta et al. were able to test both an SVM and a Random Forest Classifier on detecting a positive, negative or neutral emotional state where his Random Forest had an accuracy of 90.48%. He also found that his SVM was not as good as his Random Forest classifier and so he continued to use the Random Forest for the majority of his experimenting.

Although Gupta et al's Support Vector Machine had a poor accuracy, Khalid et al.'s SVM's does show there is some promise with this model. Two other studies were able to utilise an SVM to get higher accuracies in emotional classification, one study achieving 86.65%[42] and another study by Hui-Rang Hou et al. achieving 98.9% in emotional detection in response to olfactory stimuli when classifying into two categories (pleasant and disgust)[43]. These results do suggest that an SVM is potentially a good algorithm for emotional classification, with the second study able to achieve a very high accuracy. It is important to note that in the same olfactory study, the SVM got a lower accuracy of 88.5% when classifying into five categories instead of two. This suggests that for a lower set of outputs SVM may be significantly more accurate.

Hui-Rang's et al. study also examined using a K-nearest neighbours (Knn) algorithm, again in catagorizing emotions into two and five emotions. When catagorizing into two emotions the algorithm was able to achieve a 95.2% accuracy compared to its five emotion counterpart that only achieved 71.8%. These results are harder to interpret as Knn proved not as accurate in Gupta's et al. study, possibly meaning the higher two-bin categorization could simply be an artifact of the classifying labels being disgust and pleasantness and being more effective with this labelling than compared with other emotions like happy or sad.

2.4.2 Deep Learning Models

Deep Learning models incorporate algorithms that adjust weights of neural networks to train models to detect patterns on training data that is then tested and evaluated on testing data. Often, what makes these deep learning models unique is the layout and architecture of hidden layers. Hidden layers comprise of a set of neurons with a uniform mathematical formula with differing weights. They have shown strong promise recently in emotional classification with multiple studies testing common existing architectures as well as developing new architectures and meshes of multiple models.

Once again referencing Khalid's et al. study on attention levels, they used a neural network as well as classical machine learning algorithms with an accuracy of 88%[41], they do not mention the architecture of this neural network and as such it is hard to divulge any greater insight other than there is feasibility in utilising a neural network. Interestingly, they also use a different neural network with a defined architecture called GRU (Greater Recurrent Unit). This model is a type of Recurrent Neural Network (RNN) and has seen some popularity in sequence classification problems such as emotional classification using EEG's[44][45]. This GRU model was able to achieve an accuracy of 96.4%, five percent higher than their best classical model, the random forest. Notably, they trained this model on the raw data of the EEG collection and not on frequency band data.

Hui-Rang Hou et al.'s olfactory-based study also compared both classical and deep learning algorithms. Although unable to outperform the SVM accuracy of 98.9%, Hui-Rang et

al. were able to develop a simple Backpropagation Neural Network (BPNN) that achieved a substantial 98.2% accuracy on classifying two emotions, however, this model dropped to only 67.5% accuracy when categorising into five emotions. The drop in accuracy when the scope increased suggests that deep learning's inherent flexibility of architecture can prove promising results; the vast majority of deep learning models use backpropagation and, as such, only build on this foundation, which in this case does prove a high accuracy.

Once the scope of emotional classification grows to a multiclass classification problem, unlike the binary-category olfactory study, more complex neural networks are often developed. Shuaiqi Liu et al. introduced a Dynamic Empirical Convolution Neural Network (DECNN) that was able to achieve an accuracy of 96.44%[46], with key take aways being that Dynamic Differential Entropy (DDE) used for feature extraction in the model (as opposed to a Fast Fourier Transform or Wavelet Transformation) is a promising alternative feature detection methodology. DDE itself is only half of the feature extraction; Empirical Mode Decomposition (EMD) is also required to perform DDE. EMD breaks the EEG signals into Intrinsic Mode Functions (IMF's), then, Differential Entropy (DE) is run on each of these IMF's to output the final results of DDE. Fa Zheng et al [47] also utilised DDE in building a Linear Graph Convolutional Network that used the preprocessing method (DDELGCN). The DDELGCN was able to achieve a positive accuracy of approximately 90% on both SEED[28] and DEAP[48] datasets.

Differential Entropy is also used in many other neural networks. Zhongke Gao et al. used DE on features collected from a Fast Fourier Transformation to create their neural network, the Channel-Fused Dense Convolutional Network (CDCN)[49]. The CDCN's hidden layers are composed of a Convolutional Neural Network (CNN) followed by alternating dense and transition layers to output an accuracy of 90.63%. Another Neural Network that utilises DE is Tong Zhangs et al. Spatial–Temporal Recurrent Neural Network (STRNN) that utilised two different neural networks stacked together. Firstly, a Spatial Recurrent Neural Network (SRNN) is used on the frequency-domain (sometimes referenced as spatial-domain), which is then followed by the TRNN taking those inputs and working on the time-domain. Thus, this model attempts to utilise both domains an EEG signal outputs to get as many training features as it can. Regardless, the STRNN still only manages 89.50% accuracy. Wei Liu et al. was also able to create a model using DE that was able to get an accuracy of 91.01%[50]. Their BDAE model is composed of a series of Restricted Boltzmann Machines (RBMs) that process both EEG and eye movement signals/information individually before synthesising them together in a hidden layer to detect patterns.

The main findings from these models is the difficulty of achieving over a 95% accuracy in multiclass categorisation. DECNN is the best-performing model with 96.44% and other models struggle to reach similar statistics, both in deep learning and in Classical Machine Learning. There are also multiple other models not mentioned that sit at lower accuracies that also attempt to find optimal architectures for Deep Learning models. However, as Xin Hu et al. [51] and Wei Li et al.[24] strongly suggest, cross-subject variability is a constant issue that holds back EEG emotional recognition. A lot of the aforementioned papers acknowledge the problem; however, they do not put a focus on overcoming the challenge in their studies, thus, there is a lack of substantial evidence currently as to the effects of mitigating cross-subject variability in the field.

3

Methodology

One of the original goals of the project was to incorporate live user data collected from a Muse 2 headset and infer from that data using the developed program. However, as the practical challenges of working with different subjects and devices arose, requirements were adjusted to prioritise the research questions of the project.

The project's initial focus was on self-obtained data and utilising this data to prototype an EEG pipeline, as well as to assist in visualisation and increasing familiarity with the data inputs. These initial tests were also used to test different software solutions designed for EEG data, experimenting with both Python libraries as well as full EEG dedicated programs to help determine how to design the final program.

Once this sandbox had been explored, the project shifted to developing the EEG machine-learning pipeline. This revolved around using a Jupyter notebook to develop: a dataloader, pre-processor, feature extractor, model creator, and model evaluator. This was also when live data was switched out with a static dataset, DEAP. This change of requirements came from the results of a series of experiments run, where a machine learning model was trained on the DEAP dataset, and then live data was streamed to the trained model and evaluated. The experiments failed; however, they did provide direction for future development on the project.

A prototype was then fully developed using just the DEAP dataset on a Jupyter notebook. This prototype was used to test logic and data flow and to do rudimentary testing on the full EEG pipeline. After this proved successful, more detailed and structured versions of the program were redeveloped and built around different software architectures.

After the creation and evaluation of the different architectures' performance on static DEAP data, live inference was reintroduced. A custom dataset was created where a single person's EEG readings while they self-assessed their valence rating as they listened to music was formed. The custom dataset was then used to train a model that could, when being streamed EEG data from a Muse, be evaluated live.

3.1 Hardware Setup and Usage

For live data, a Muse 2 headset was used. These headsets are commercially available and provide a cheap entry-level into scanning EEG signals. As well as EEG, they also collect other data that can be potentially used when creating algorithms or discussing data, notably heart rate and movement. Movement tracking will be able to assist in potential artifact removal, such as the MNE Python library, which uses electrocardiography (ECG) artifact removal. This is due to subtle movements producing artifacts in the EEG data when recording.
The Muse 2 is a dry node headset, meaning that it requires minimal preparation (dampening the sensors and scalp) before being put on. This greatly improves its portability and setup time.
The Muse 2 does sacrifice breadth for its accessibility. The device only collects data from four electrodes: TP9 (Left Temporal), AF7 (Left Frontal), AF8 (Right Frontal), TP10 (Right Temporal), as seen in fig 3.1, where other devices can range from collecting 32 to 128 different electrodes. Although having more data inputs would likely lead to the creation of more effective deep learning models, the price and accessibility of these models are not feasible. Additionally, the Muse's lower cost could also potentially mean less accuracy in readings compared to high-end devices; however, preliminary tests have found the data sufficient and workable.

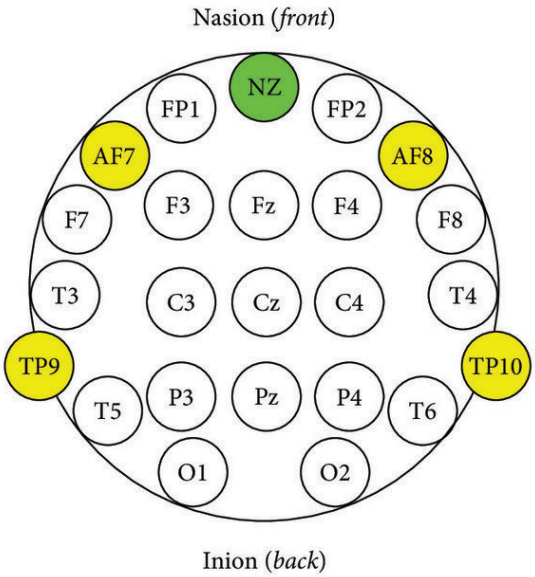


Figure 3.1: The sensors of the Muse 2 headset are highlighted in yellow. "NZ" dictates Nasion, which is used for device placement[52]

The Muse 2's biggest drawback for this project was its difference from the device used to capture the DEAP dataset participants' data. The DEAP dataset collects 32 electrodes; however, none of the 32 they tracked were locations on the scalp that the Muse 2 has electrodes, meaning that none of the 4 datastreams from the Muse could be assigned a datastream from any of the 32 electrodes from a DEAP participant. This issue of cross-device variability prevented any machine learning algorithm from detecting patterns in the same data streams. A model trained on the DEAP dataset will be looking for the patterns in the MUSE dataset that align with the streams from the DEAP dataset, and as there are none, the models fail to



Figure 3.2: An image of the muse secured on someone's head. Note how the device covers the frontal and side regions of the brain only.

make any informed decision.

3.2 Software Tools

Muse/EEG Software Options

Muse does not have supported software for extracting raw data from the device, and as such, other alternatives have to be evaluated. There are many different public GitHub repositories for extracting the data from a Muse device through a Bluetooth connection. Often, these programs run various Python scripts that allow the data to be extracted and sometimes pre-processed. However, the unreliable nature of the developers of these Git repositories and the lack of these repositories having a large user base suggest resources that are unlikely to work.

Instead, a software called 'Mind Monitor' has been used in many different published papers and has a significant amount of research backing. It is positioned to assist in the research of EEG and is designed for use with the Muse. For these reasons, I decided this would be how I extract the data as it allowed raw data extraction, as well as built-in feature extraction for testing. Mind Monitor allows for various settings to be tweaked, including frequency, sampling rate, and recording format. These values are all essential for formatting the data for machine learning in subsequent steps. It also provides a notch frequency filter, which filters out the frequency of power lines, a known artifact that is often removed in pre-processing.

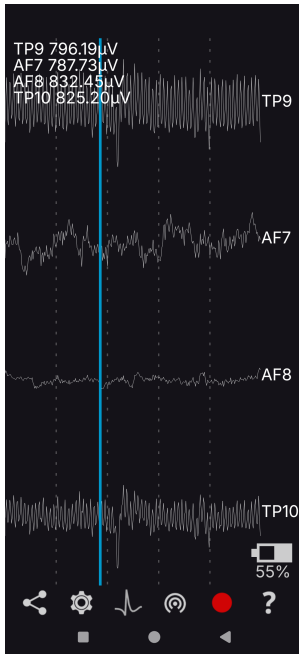


Figure 3.3: The RAW EEG data being streamed from the MUSE 2 to Mind Monitor

There are also various ways to graph the data. Mind Monitor provides a .csv file of your data as well as a script that graphs it into frequency bands. The Mind Monitor CSV file provides numerical data for each timestamp representing the recorded sampling rate (by default 256Hz). It provides each node's Fourier band as well as the raw data, gyroscope, and heart rate. A small excerpt of this numerical data can be seen in fig 3.4. Additionally, for more focused graphing, the MATLAB extension EEGLAB provides a vast amount of graphing features, most outside the scope of the project, but it does provide a strong amount of custom graphing where needed. This software was used to provide initial testing and visualisation of the data.

Gamma_T	RAW_TP9	RAW_AF7	RAW_AF8	RAW_TP1(	AUX_RIGH	Acceleron	Acceleron	Acceleron	Gyro_X	Gyro_Y	Gyro_Z	PPG_Ambi	PPG_IR	PPG_Red	Heart_Rat
0.1355	802.637	803.846	783.7	807.473	866.703	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	751.868	795.385	788.938	758.718	711.978	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	774.835	792.967	788.938	768.791	698.278	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	851.392	799.414	766.777	836.887	852.198	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	859.853	805.055	775.641	850.586	892.088	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	811.502	807.07	784.908	815.128	818.352	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	755.897	804.249	764.762	766.374	682.564	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	774.835	797.802	767.985	776.447	784.506	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	853.407	794.176	776.85	843.736	1168.5	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	866.3	801.832	770.403	857.033	1066.56	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	815.934	801.429	782.088	813.114	830.037	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	757.509	788.535	792.967	767.582	796.996	-0.13208	0.0061	0.98621	3.1028747	-2.73651	-3.1926	443223.44	6.0297728	4.2486593	63.293
0.1355	767.18	790.147	782.894	778.864	767.582	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293
0.1355	836.484	800.22	772.015	831.245	740.989	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293
0.1355	859.048	796.593	774.029	859.451	727.289	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293
0.1355	835.275	801.429	776.85	838.901	818.352	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293
0.1355	770.806	805.458	762.747	780.476	797.802	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293
0.17857	748.242	803.443	772.821	765.165	754.689	-0.15826	-0.00806	0.97723	2.7589416	0.4187	-6.19827	443223.44	6.0297728	4.2486593	63.293

Figure 3.4: A small section of the Mind Monitor CSV numerical data.

EEGLAB also provides all the tools required for pre-processing. It allows for various band-pass filter options, blinking and movement artifact removals, noise reduction and the infrastructure to introduce custom pre-processing if required. However, due to its complexity,

a significant amount of redundant options and a hard-to-understand user interface, other options were looked into instead. MNE Analysis and Visualisation is an open-source Python package that allows all the necessary tools for developing the EEG pipeline, and combined with scikit-learn, another open-source Python library, for developing artificial intelligence models, as well as training and testing them. Building the program solely in Python was a much simpler and accessible method to develop the program.

Initial tests were conducted using EEGLAB and Mind Monitor to determine if immediate patterns could be seen in a small group of participants listening to emotive music while their EEG data was collected by the Muse. The results of this can be seen in the Preliminary Results section of the paper.

3.2.1 Workflow Software

The program was developed in Visual Studio Code using Python. Python was chosen first to accommodate the two aforementioned libraries, MNE and SciKit-learn, but also as the prototypes of the program were done using Google Colab's Jupyter notebooks. Google Colab allowed the fast development of small parts of the code. First, it was used for importing Muse and DEAP data into a workable format, and then for visualising this data and collecting preliminary results. VS Code was then used once a more complex structure was required, specifically for the testing of specific architectures. Git was used throughout the process for version control, as the program was to have multiple iterations with different architectures. Git was essential for keeping these versions all accessible and safe. To connect the Muse 2 device through Bluetooth to stream data, the BlueMuse Git Repository software was used. Additionally, to interface with the Lab Streaming Layere (LSL), pslsl was used.

3.3 Prototype Pipeline Development

An initial prototype was developed using the EEG data from both the Muse and the DEAP dataset, once it was successfully imported into the Google Colab notebook. The prototype was designed to allow the creation of each section of the pipeline one after the other; this pipeline being:

- Dataload - The EEG data is stored in the system
- Pre-process - Pre-processing on the data, such as Independent Component Analysis (ICC) and band filters, can be executed on the data.
- Segmentation - The data is broken into epochs, where the DEAP dataset participants were watching emotional stimuli.
- Feature Extraction - The preprocessed data then has extra information extracted using a formula
- Model Creation and Evaluation - Simple models are built and tested on the data

The prototype acted as an MVP for the project and served as the baseline architecture for comparison. The above pipeline is upheld in all versions of the architecture and ultimately uses DEAP data for the entire system, from training a model to inferring on data. A detailed description provides context for how this pipeline is formatted and reorganised through other architectures.

3.3.1 Dataloader

The dataloader imports EEG data from a source to be used in the program. The program was originally designed to have a degree of flexibility in the format of the data it would import; however, due to the larger difference in dataset preparation than originally planned, the program was ultimately only able to take in RAW DEAP participant data as a binary data file ('bdf'), or a .csv file from Mind Monitor.

One major issue faced during this stage that led to the limiting of the scope was the lack of standardisation used across EEG datasets and applications; there was no consistent naming scheme for EEG electrode channels, which prevented a smooth creation of a universal data structure for the program. It was crucial to know which data belonged to which electrode, and as such, if one dataset has 'RAW_TP9' and another 'TP9raw', for example, the program was unable to map both to a TP9 electrode. Future solutions can be easily implemented into the program to solve this issue; however, time constraints meant it was not a high priority for this project.

Additionally, naming problems arose within the DEAP dataset itself, as the dataset was created in two different locations using different hardware. A third of the participants have a different ordering of electrodes in their 'bdf' file, necessitating work to rearrange the electrodes to a universal consistency. This extra work for the dataloader strictly for the DEAP dataset was pushed to a low priority due to time constraints and a lack of impact on the research questions of this project. Future work could see adjustments made here to access these participants' data; however, it would be at risk of having the program become too DEAP-centric. There is likely a better alternative that would solve this issue, alongside making the dataloader more practical for other datasets.

In addition to extracting the data from the files provided, the dataloader also saves the emotional labels of the participants. As discussed in the literature review, labels are collected from just the valence data in the dataset to be used for a dimensional layer during model creation and evaluation.

3.3.2 Pre-processor

The pre-processing stage of the pipeline underwent the most iterations through prototyping and into the final program. Pre-processing is required to format the data in the best way for machine learning. There are many different ways to pre-process data, and the program allows for, bandpass filtering, downsampling, and Independent Component Analysis (FastICA, Piccard, InfoMax).

The program is designed to allow for flexible testing and allow for multiple configurations, and as such, each of the above settings can be configured to produce comparative results against other configurations. Notably, however, the order of the operations for the pre-processing stage is relevant and cannot be organised in a different order. Various issues can come from performing some tasks before others. For example, it is ideal to run a bandpass filter over the EEG signal prior to downsampling, as if done inversely, aliasing issues can appear in the signal as it may be misrepresented due of oversimplification.

In the pre-processor, the order of operations is as follows: Notch Filter, Bandpass Filter, Downsample, ICA. This order looks to remove artifacts when possible without hindering the following steps (or having the artifacts nestle deeper into the signal, like aliasing problems). A notch filter, although irrelevant if a bandpass filter is performed, removes persistent static from the mains power, followed by the bandpass filter that cuts off all signals above 45Hz and below

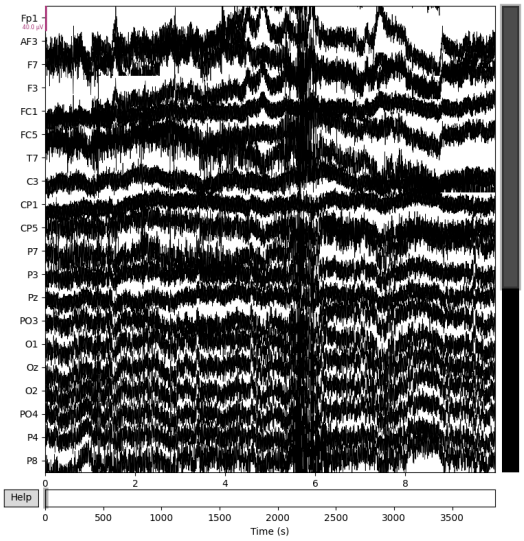


Figure 3.5: Snippet of raw EEG data of a participant, no pre-processing has been performed. Sample rate is 512Hz

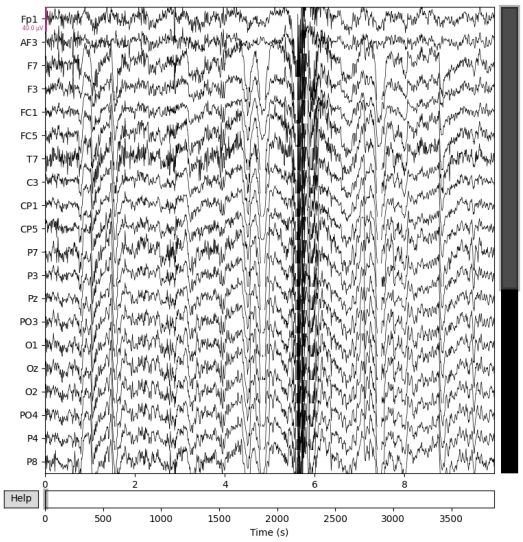


Figure 3.6: The same data now pre-processed with a bandpass filter, down-sampled to 128Hz, and FastICA performed

1.5Hz, then downsampling resamples the data signal to the designated samplerate before independent component analysis attempts to separate the signal into its core components. Additionally, during ICA, electrooculogram artifacts, often from blinking and eye movements, are attempted to be removed.

The algorithms for these preprocessing methods are sourced through the 'mne' Python library.

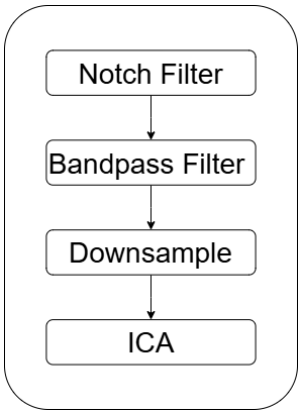


Figure 3.7: The order of operations in the pre-processing pipeline

3.3.3 Segmentation

Segmentation is the act of separating a block of data into epochs that can be fed into a machine learning algorithm. In essence, the more segments of data an algorithm can learn from, the more likelihood that patterns can be found across new data. As the DEAP dataset is recorded as a long stream of continuous EEG data while a participant is subjected to various emotional stimuli, our machine learning wants to be able to break down the participant's data into blocks

of specific emotional signals (high or low valence) so that we can label these blocks with the correct emotional label before learning on them. This is the role of the segmentation class.

The DEAP dataset, although being a continuous stream of electrode recordings for a single participant, does also record events. The events tell us what is happening at points in the stream. For the DEAP dataset, this includes events for: experiment start, baseline recording, and video start. Among other labels, these are the key events that allow our segmentation class to take meaningful snippets of a participant while they are watching a video.

The project also uses sliding windows with configurable length and overlap. The baseline for these values is a 4-second window (an epoch of 4 seconds) with an overlap of 0.75. Meaning, for each epoch, we include 75% of the previous epochs' data. See figure 3.8

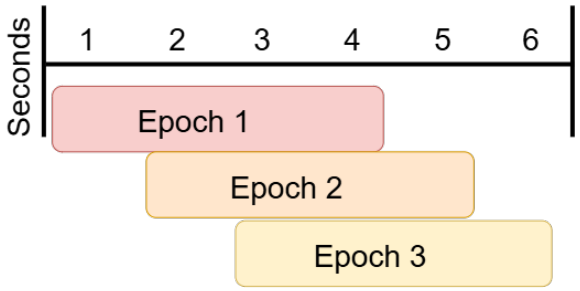


Figure 3.8: Three sliding windows of length: 4, overlap: 0.75.

Each participant provides around 57 epochs with the default settings, and although some epochs will be dropped due to poor quality, doing this for 22 participants nets around 50,160 epochs of data for the machine learning algorithms as each participant provides around 2280 epochs. Different configurations will provide other results, but the segmentation class is able to create large datasets regardless.

The other task performed by the segmentation class is associating the correct label to each epoch. As the program creates a dynamic amount of epochs based on the configuration, there must be a method that can dynamically assign a label to each of these epochs. The program creates a label array equal in length to the number of epochs. Each label corresponds directly by index, meaning that for every epoch at index i in the epoch list, there exists a label at index i in the label array (formally defined as $f(e_i) = \ell_i$). For example, the n-th epoch in the epoch list corresponds to the n-th label in the label list.

3.3.4 Feature Extraction

The original scope for the program was to include at least two methods of feature extraction that can be used in the program. A Fast Fourier Transform (FFT), and a Wavelet Transformation (WT). Ultimately, for the prototype, FFT was included and built in and WT was left out as it did not assist the answering of any research questions and would require a significant time contribution to developing it.

The feature extraction class takes in all of the segmented epochs and labels and breaks down the epochs from their EEG data into frequency bands. These bands are as defined by the National Library of Medicine; however, they can be adjusted to accommodate other academic definitions of FFT bands.

As the DEAP dataset has 32 electrode channels, each channel is separated into the five frequency bands providing 160 features for each epoch. The class returns this information as

a 2-dimensional array (epochs, features)

3.3.5 Model Creation and Evaluation

Model creation is handled by the 'model.py' file and houses all the model classes. The program can create support vector machines (SVM), Random Forest Classifiers, K-nearest neighbour (knn) algorithms, as well as a simple multi-layered perceptron (MLP) neural network.

The models all follow the same input, a 2-dimensional array from the feature extraction, and therefore, additional models that take in an array (epochs, features) can be added in easily. This covers most traditional machine learning approaches. There were plans to also include more complex deep learning algorithms, such as convolution neural networks, or recurrent neural networks; however, these models require an extra time dimension to work. Additionally, they also require a fair amount of changes to the program's environment configuration, which would take additional time and would also utilise different resources from the aforementioned traditional algorithms. As the program aims to prioritise testing comparatively, the performance of the software architecture, and not specific model performance, it made more sense to focus on analysing and comparing the effectiveness of the same resources.

Model performance and evaluation are still important metrics, and the program allows for the creation and testing of multiple of these configurations at once. As such, a wide range of configurations for each of the traditional models is tested and discussed in the results. All the models produce the following metrics for analysis: accuracy, recall, precision and an F1-score.

3.4 Architectures

In the initial outlining of the project and discussion of what architectures should be used key Architecturally Significant Requirements (ASR's) were established and prioritised. With the goal of the project being EEG-based emotional classification, the key ASRs decided were performance efficiency, modularity, scalability, commonality, and reproducibility. These ASRs, although generic, led to the decision to test a layered architecture, pipes and filters, and a microkernel. Additionaly, the prototype Jupyiter notebook acts as a separate non-structured architecture.

3.4.1 Layered Architecture

The layered architecture is commonly used in industry and, as such, often ends up as a default architecture for projects. This provides a good baseline architecture to evaluate, as the commonality of the approach may lead to significant inefficiencies in performance, which will be ideal to highlight with results.

The architecture lends itself simply to the re-organising of the original prototype Jupyter notebook. The above pipeline is essentially sorted naturally into the common layers found in the architecture, which are: Application, Domain, Infrastructure, and Presentation layers.

Although these layers are not required for the architecture to be classified as layered, they are the common foundations for the architecture. The program is separated into these layers for simplicity and to make the architecture as traditionally layered as it could be.

The presentation layer includes the code for creating visualisations of the data, such as confusion matrices and displaying performance metrics, and also the main.py file, which serves as the entry point of the program, the initial input of data, and preparation and running of the pipeline. This is where many parameter changes and configurations for the program can be updated and experimented with. Within the application layer, we have our pipeline code, which handles the primary constructors of our key EEG pipeline as well as organises the logic for the domain layer. This layer calls the domains in the correct order of operations and ultimately passes the final results from the models' domain to the presentation layer. The domain layer itself houses the core functionality of the program. Located here are the dataloader.py, preprocessor.py, feature_extraction.py, and segmentation.py files that handle the code for their respective tasks, as well as models.py, which includes all the model classes used in the program. Lastly, the infrastructure layers handles the loading of valence labels, and if there were to be any other major data storage requirements, they could be appended here.

Each layer is structured to only ever communicate with adjacent layers to uphold a true layered approach. Figure 3.9 provides a visual overview of the architecture structure.

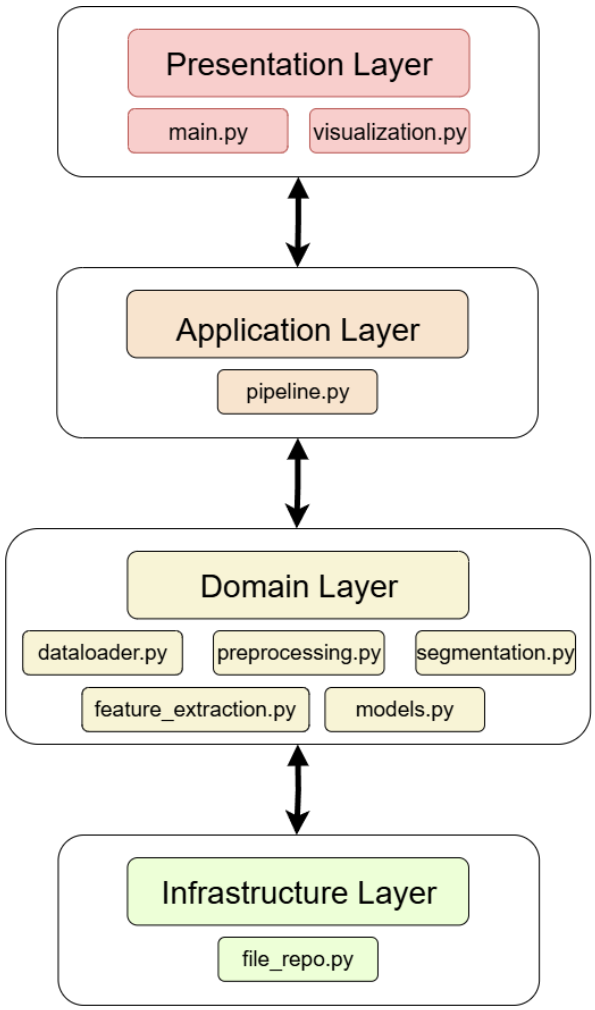


Figure 3.9: A visual depiction of the file structure of the program represented as common layered architecture layers

The architecture does allow for simple modularity; it is easy to build in new models, for example, and the logic flow would remain the same if future work led to implemented

changes in the segmentation and feature extraction domains to allow for multidimensional data to allow for more complex models. The logic of the EEG pipeline is concrete, which lends itself well to the architecture; the only major changes to the program would be in the data flow itself and thus could be added in simply. The original prototype MVP transferred nicely into the layered architecture because of this, when the need to make changes to the codebase came it was very easy to incorporate and the additional code required for preparing results was also neatly added in. More models were also implemented after the original prototype and once again, this architecture made that significantly easier. Ultimately, the layered architecture was much better than working in the unstructured Jupyter notebook, providing significantly better modularity, maintainability and reproducibility.

3.4.2 Pipes and Filters

To transform the prototype into a pipes and filters architecture, the first stage was identifying all the independent filters of the program. This was achieved by breaking down each of the major stages in the pipeline —dataloader, pre-processor, segmenter, and feature extraction —into their core functionalities. The filters themselves should only ever perform one transformation on the passing data and should take an input and output of the same structure to allow for mass modularity.

The first key challenge to handle with the architecture is the data structure of the input and output. By having a common data structure through the architecture, the program can be modularised to a point where any filter could plug into any other filter, leading to the core dataflow logic being the only barrier to error. Initially, it was planned that passing the raw EEG data object through the program would be sufficient, as each stage works on specifically transforming this data; however, this resulted in errors early on in the pipeline when label collection was managed. The 'labelLoad.py' filter's goal was to get the valence labels from the Excel spreadsheet and store them in an array. This led the filer to not needing the raw data as an input or an output, and although the program could be written in such a way that still passed the data through the filter, the label array also had to be then passed through the rest of the filters until it was required leading to every filter being required to return the raw data and label data. Two solutions were drafted up for this problem, the first being to create a second pipeline that strictly acts with labels, which added a significant amount of overhead and potential problems down the line of lining up the pipelines, or the alternative option to create a custom datatype known as a packet that was designed to be passed through every filter and store all the relevant info of the program as it went by.

Due to the first option only having one filter, the loadlabel filter, it didn't make sense to create a whole parallel pipeline for it. The packet solution was also simple to implement and significantly more maintainable in the long run. Ultimately, the packet class was created that housed the core data the pipeline would need to transform and update as it ran. See figure 3.10

With the packet being passed through the pipeline, allowing filters easy adjustments only to the relevant parts of the dataflow, the pipes and filter architecture became a much simpler architecture to build. Although there may be potential methods to optimise further by removing some of the fields in the packet, and by offloading some of the work to the start of the program where initial setup is made, the architecture does work smoothly and as intended, allowing for filters to be added or skipped to allow for the custom configurations the program aims to deliver. A quick comparison to the layered architecture confirms the pipes and filters more modular purpose; within the layered architecture, when code wanted to be

```python
class Data:
    """
    Holds all shared EEG processing data for the pipeline.
    filters transform/modify some part/s of this packet and returns it.
    """
    def __init__(self, filepath=None, excelFile=None, participant=None):
        self.raw = None          # MNE Raw object
        self.events = None       # numpy array
        self.labels = None       # array
        self.participant = participant  # int
        self.features = None     # feature matrix
        self.filepath = filepath     # path to EEG data file
        self.excelFile = excelFile   # path to labels file
        self.X_segments = None      # segments of EEG data
        self.y_segments = None  # labels for each segment
```

Figure 3.10: An extract from the pipes-and-filters architecture showing the custom data structure used to handle passing data.

bypassed for certain configurations, the program would still need to pass over the unwanted code, if just to ignore it, whereas, with pipes and filters, the program can remove the code that the layered approach would still see and bypass, saving resources as the code is gone from the pipeline. At a low level, this implies that pipes and filters should be a more efficient architecture when testing resource usage.

Having the program atomised down into separate one-purpose transformations also greatly increases the code readability, maintainability, and testability of the entire program. Every section of the program can be very quickly and easily isolated and independently tested, abstracted away from the rest of the program. As each filter takes in the same input and provides the same output as the other, the difficulty of creating a test environment is heavily lowered. Changes can easily be made and quickly tested, and new filters can be added and quickly tested in conjunction with other filters to test compatibility.

3.4.3 Microkernel

Shifting the prototype program into a Microkernel had some minor challenges, especially in terms of control flow when evaluating models after the initial pre-processing pipeline was finalised. The first step was to create the kernel and provide it with the core functionality it would need, such as being able to register plugins to itself, and then to execute the list of plugins and run them all. The kernel developed here does this linearly in the order the plugins are registered. Each plugin is registered with a name needed to identify the control flow later on when running the plugins. The section of code for the kernel can be seen in figure 3.12

The next functionality added were the dataloader plugin and the labelloader plugin. These plugins introduced the first challenge with the architecture. Similar to pipes and filters, the codebase needs to be designed in a way where all the modules take the same input and produce the same output; since the architecture's command flow is designed to be flexible and not have a strict order of operations, inputs and outputs are naturally more ambiguous. In the pipes and filters architecture, this was solved through the use of a packet that was passed that housed all the data that would be needed through the program. For the microkernel, a similar data storage is passed through the program's plugins; a Python dictionary. The implementation of the dictionary felt incorrect as it seemed to lean into pipes and filters, where the program looked to be becoming a series of operations where the dictionary was passed to each plugin, transformations were done, and then the data would move on. However, the key difference

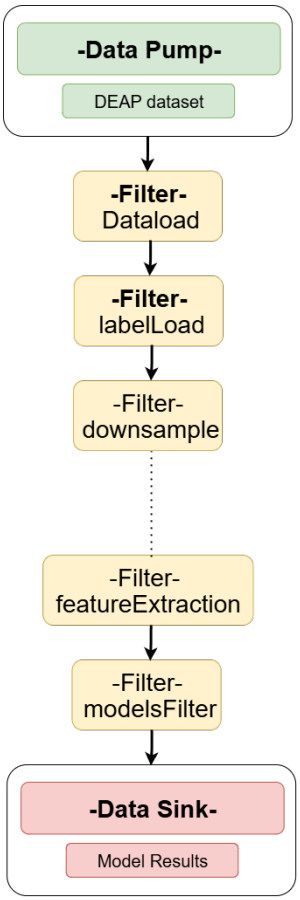


Figure 3.11: A simple overview showing the architecture of the pipes-and-filters program and some example filter logic

```
microkernal.py > Kernel > execute
1   class Kernel:
2       def __init__(self):
3           self.plugins = {}
4
5       def registerPlugin(self, plugin, name=None):
6           name = name or plugin.__class__.__name__
7           self.plugins[name] = plugin
8
9       def execute(self, data):
10          data = data
11          for plugin in self.plugins.values():
12              data = plugin.execute(data)
13          return data
```

Figure 3.12: The kernel class for the microkernel architecture, this houses all the plugins attached in the program and holds functionality to run them

between the two is ultimately the command flow, where in pipes-and-filters, the architectural pattern dictates that the data passes through the filters one after another until it reaches a sink, an example of data-driven command. In comparison, the microkernel architecture instead has all the control in the power of the kernel; the data isn't passed between plugins, it is pushed and pulled through the kernel until the kernel dictates it to stop, thus being centrally managed. This is a good highlight of the added flexibility in the architecture, but also where the added overheads of communication lie.

After the creation of the data and label loader was complete and the kernel was successful in registering and running them, configuring new plugins was simple. However, the next barrier of the architecture arose when developing the preprocessing plugin, where parameters needed to be passed into the plugin. As the program must have highly configurable parameters, handling different parameters in the plugin was unavailable. Ultimately, as the plugins were only included in the system when registered, the parameters would also need to be decided when the plugin is initially registered in the kernel. Further complications arose as parameters set like this could only be set statically in the '__init__' function, and not at runtime in the execute function, as this function call is the same for all plugins. This is one downside to the microkernel architecture; although promoting consistency, having each plugin follow the interface reduces flexibility in the program. The solution was to have the parameters passed into the initialising function on registration, and then in the plugin, make them accessible to the execute (and other) functions. This is done by creating config dictionaries in the main.py file and then passing the specific configs into the register plugin calls seen in figure 3.13. Once the config is initialised in the plugin, any of the needed parameters are easily accessible

```
preprocessConfig = {
    "notch": True,
    "bandpass": True,
    "downsample": True,
    "ica": True,
    "downsample_rate": 128,
    "lFreq": 1.5,
    "hFreq": 45.0,
    "icaMethod": "fastica",
    "randSeed": 42,
    "n_components": 0.999999,
}

kernel.registerPlugin(PreprocessPlugin(config= preprocessConfig), "preprocess")
```

Figure 3.13: An example preprocessing config for the preprocessing plugin. The config is created and passed into the plugin on initialisation.

Feature extraction and segmentation plugins followed the same methodology as the previous plugins, with configs being passed in during initialisation when needed. The development of the model plugin led to issues; the plugin was initially added with the other plugins, and as such, it would always run once at the end of the kernel order. If multiple models needed to be tested, the plugin would only be able to run once and not for each model that existed. There was a possible solution to pass a model config into the plugin and iterate over the config inside the plugin; however, this felt counterintuitive to keeping the main command centre as the kernel. Instead, the solution implemented was to create a new kernel that just had the model plugin registered and iterate over the kernel for each config in a config list, passing that config list into the plugin and printing the results. This achieved the wanted outcome.

3.4.4 Dataflow Diagram of Architecture Testing

The dataflow diagram shown in fig 3.14 provides an overview of the system at the time of architecture testing; it excludes all live inference processes.

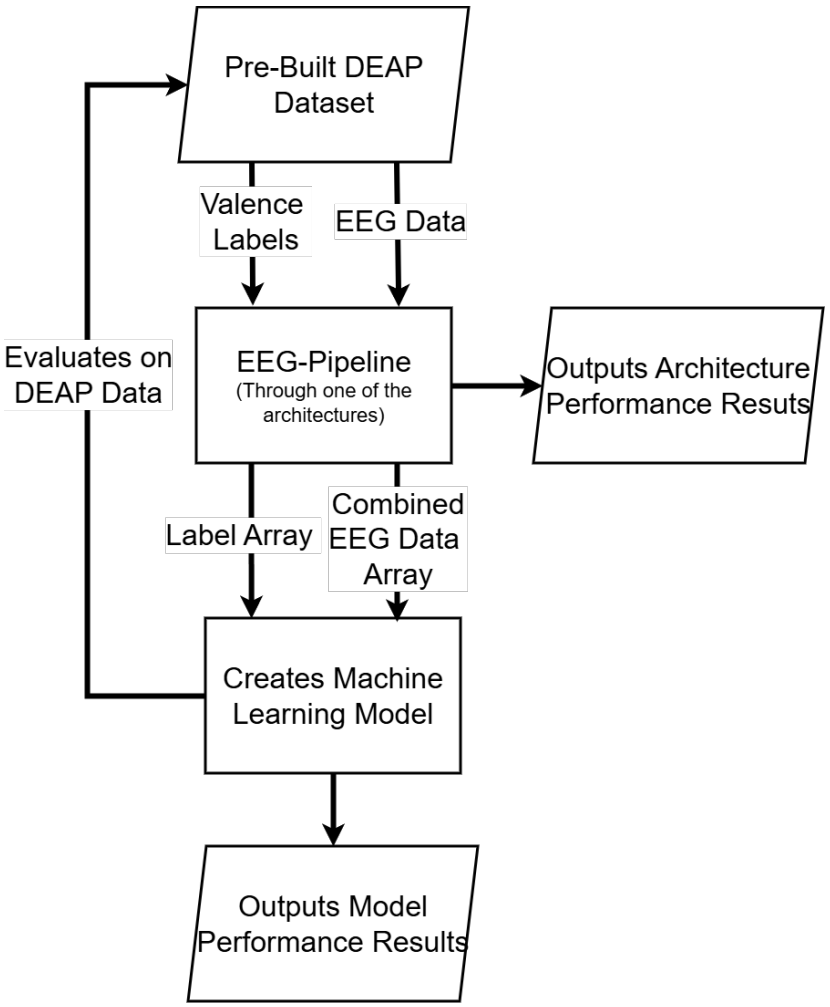


Figure 3.14: A dataflow diagram of the entire system used to test and evaluate the different architectures

3.5 Live Inference

After all the aforementioned architectures were evaluated on their performance, a seperate script was developed to test live inference. Live inference in this experiment comes in two stages.

1. Training a model on static data.

2. Streaming data from a Muse that the model looks at and makes predictions on.

The predictions the model would make would be a binary decision, where 0 meant the participant was feeling unpleasant (negative valence), and 1, conversely, meant the participant was feeling pleasant (positive valence).

Cross-subject and Cross-device variability have to be non-issues for live inference to be functional. As per early testing, with Muse data failing to work on DEAP-trained models, and for extensive research on how different individuals' brains are, the dataset was collected by a single individual using a Muse 2. The live inference was also performed on this same individual using the same Muse 2.

3.5.1 Initial Testing

Once the initial prototype was finished, prior to exploring the performance of other architectures, tests were done to check the feasibility of adding live inference into the program. A model was trained on the DEAP dataset using the prototype, where psylsl and BlueMuse then allowed EEG data to be streamed into the program. These tests yielded poor results. The inference consistently predicted only '0', showing real-time cross-subject and cross-device variability. However, this confirmed the proof of concept of the possibility of streaming live EEG data into the program and having a model perform inference on it.

3.5.2 Developing the Dataset and Script

The first stage was to solve these variability issues through the creation of a single participant dataset collected on a local device. The participant chose 10 songs, 5 songs to be positive, and 5 to be negative. This personal bias allows the dataset to be further customised to the individual and can better get a sense of when the participant is feeling more unpleasant or pleasant when listening to new songs. Each song was trimmed to two and a half minutes so there was equal training data of both positive and negative valence songs in line with typical machine learning classification practice.

Original expectations were to expand the program of one of the architectures to include live inference in the pipeline; however, this was not needed as the live inference was ultimately designed as a separate Python script entirely. This was for simplicity, as the main research question regarding live inference was more on the possibility, not on its integration into a system.

Based on the results gathered, a Random Forest (with 500 trees) model was used for live inference. The model was trained on the custom dataset of 10 songs and then loaded into the live inference script. The Muse 2 was then worn and connected to the laptop running the script through BlueMuse, where the live EEG data was then able to be accessed through psylsl in Python. This data was then preprocessed with the same configuration as the dataset, and the model was inferred on windows of the EEG data as it came in, similar to the segmentation module of the pipeline. The program looked at these windows and predicted the valence of the EEG signals. The script worked successfully, with the script predicting 0 or 1 when the participant was listening to music.

3.5.3 Dataflow Diagram of Live Inference Setup

The dataflow shown in Figure 3.15 presents an overview of the entire system used when performing live inference on a model trained on the Muse 2 custom dataset, using BlueMuse and LSL to connect a Python script to pass data into a laptop.

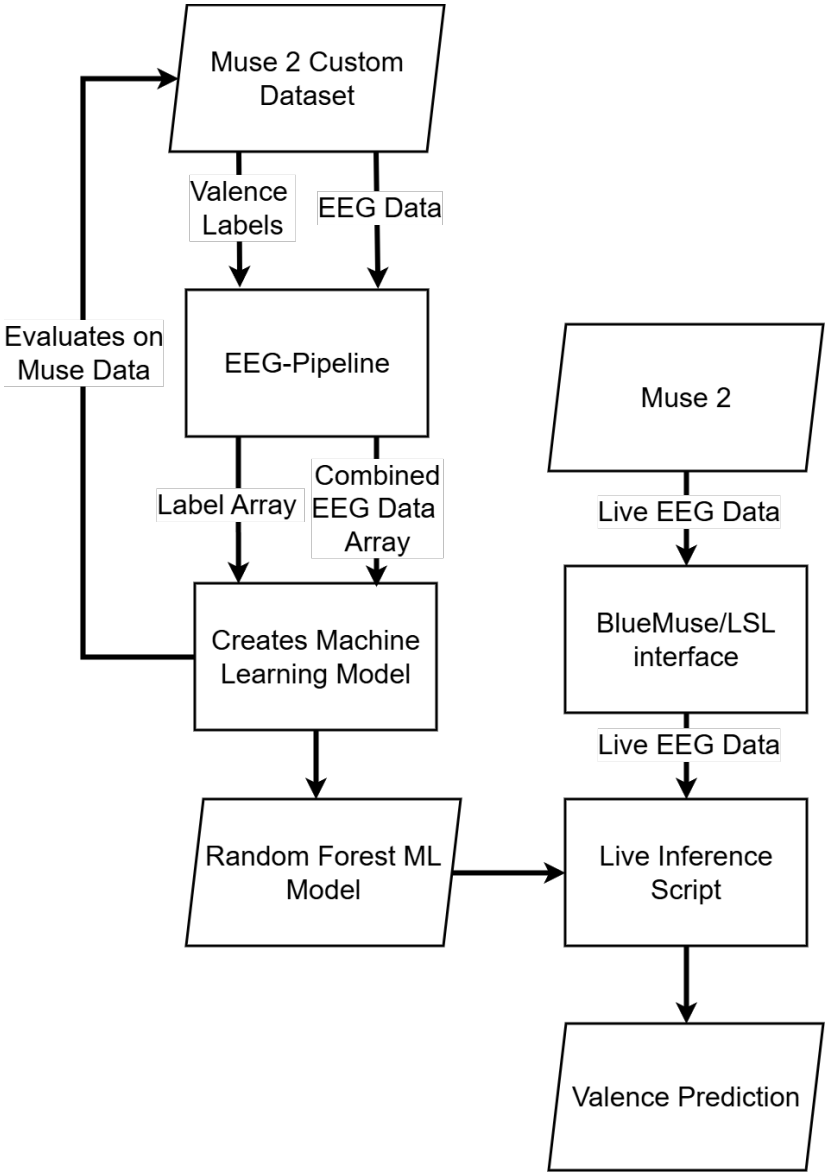


Figure 3.15: A dataflow diagram of the entire system used when testing and performing live EEG data inference

4 Preliminary results

4.1 Overview and Preface

These preliminary results were captured to prove and justify the data flow and data visualisation of the methodology. Two participants listened to relaxing music for 120 seconds before being played a song with positive valence, then they listened to another 120 seconds of relaxing music before a song with negative valence was played. Notably, for the preliminary results, the EEG and heartbeat of the participants were also recorded. There is also a section on showcasing EEG artifacts and the removal of them.

The setup can be seen in figure 4.1, participants had their eyes closed and music playing from a speaker system plugged into a computer. The data was recorded on the mobile application Mind Monitor, and the results, once saved, were sent to the computer via email.

4.2 Comparing Frequency Bands

First Period Of Relaxing Music

First, the participants listened to relaxing music for 120 seconds. Participant one's (P1) frequency bands can be seen in figure fig:relaxing1

This data can be difficult to read and attempt to interpret due to the noise of having every frequency band present, therefore, subsequent diagrams will be broken down into relevant waves. It is important to note, however, the relationship the waves have with each other, such as the potential artifacts in spikes or reactions that were strong enough to affect each wave similarly.

Looking at P1's data, from the start to the end of listening to the relaxing music, the only wave with noticeable change is the gamma wave. Seen in figure 4.3, we notice an ultimate rise to around 71Hz, suggesting a spike in deep concentration and brain activity. Compared to the second participant (P2) in figure 4.4, whose gamma waves decrease down to around 37, we can see a stark difference in the result of listening to relaxing music, one that suggests that what was able to relax P2 only heightened P1's concentration. This is a clear example of how


;

Figure 4.1: A picture of the setup with the Muse headset fitted and the data being recorded on a phone using Mind Monitor

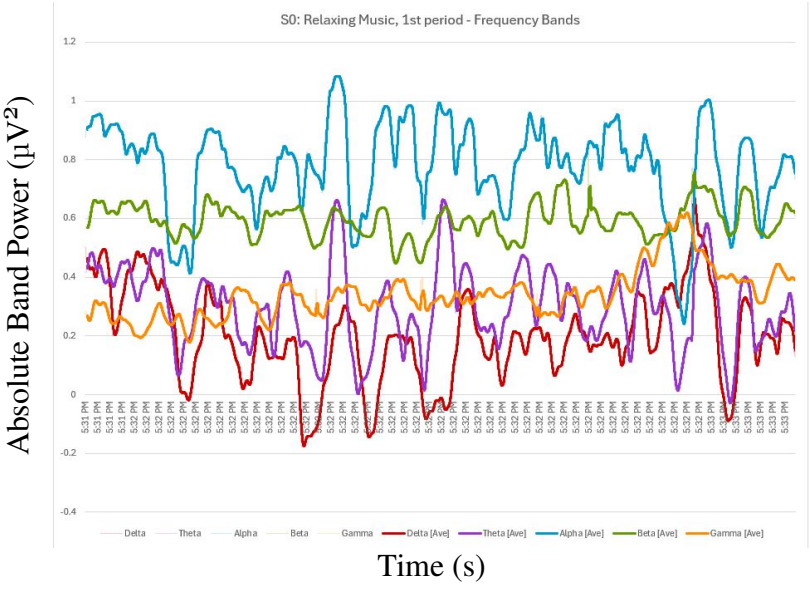


Figure 4.2: A 120-second snippet of EEG reading while the participant listens to relaxing music.

cross-subject variability can manifest in these studies, there is a large difference between the two reactions of the participants. For one person, the music was much more relaxing and was able to relax them to some degree, yet someone else's brain reacted in a way that heightened concentration and lowered their relaxation levels. The hope is that AI models will be able to find more complex patterns in these data extracts that may suggest similarities that are hard to view looking at them graphed.

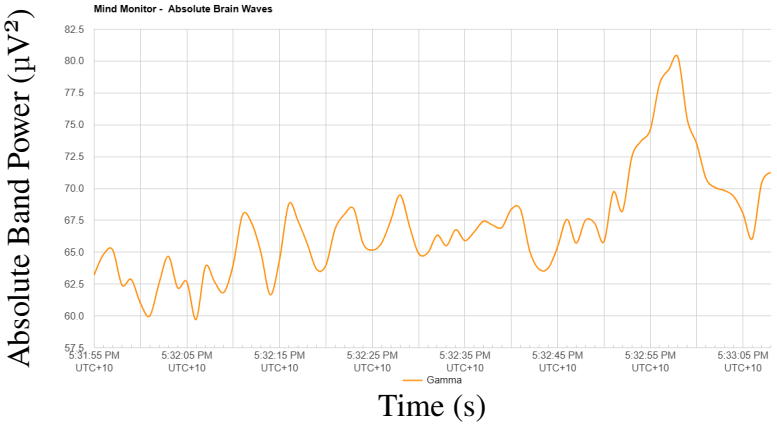


Figure 4.3: P1's gamma wave over time increases as they listen to relaxing music.

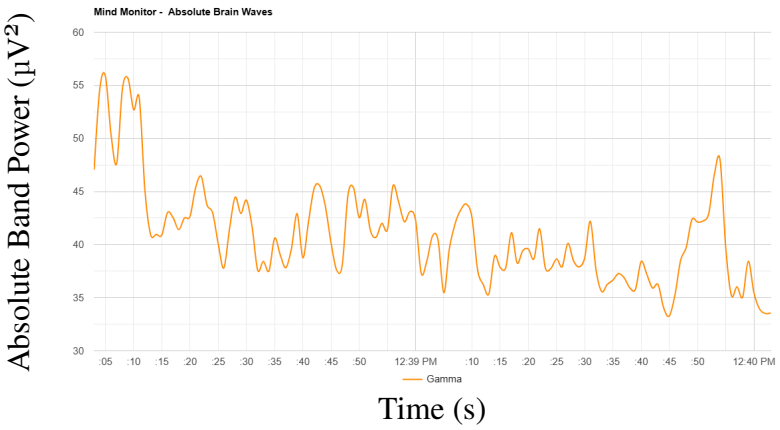


Figure 4.4: P2's gamma wave over time decreases as they listen to relaxing music; contradicting P1's

Additionally, there were no noticeable heart rate differences after the first round; both participants' average heart rate was stable and lacked any markers of relaxation or activity.

Positive Valence Music Reaction

Following the relaxing music, both participants were played music with positive valence. Continuing to look at the gamma waves of the participants, we see both people having a decrease in wave activity, suggesting entering a more relaxed state of less cognitive thinking. Cross-subject variability can quite easily be seen in the rates of activity; however, P1 in figure 4.7 floats around 65Hz, whereas P2 in figure 4.8 descends to around 35Hz. A healthy

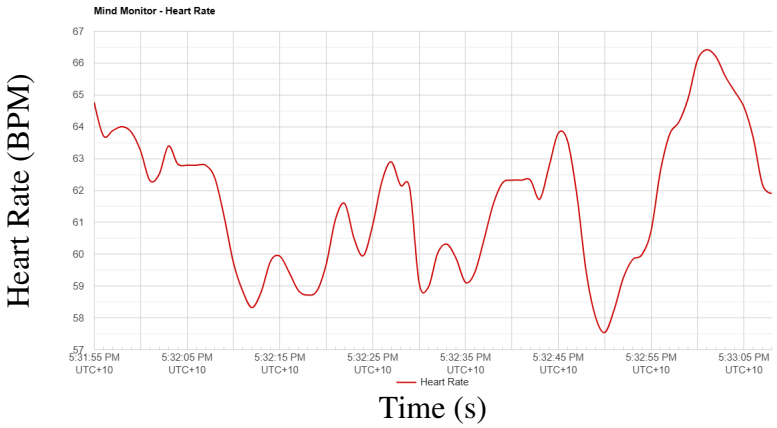


Figure 4.5: P1's heart rate does remain stable also suggesting a lack of relaxation

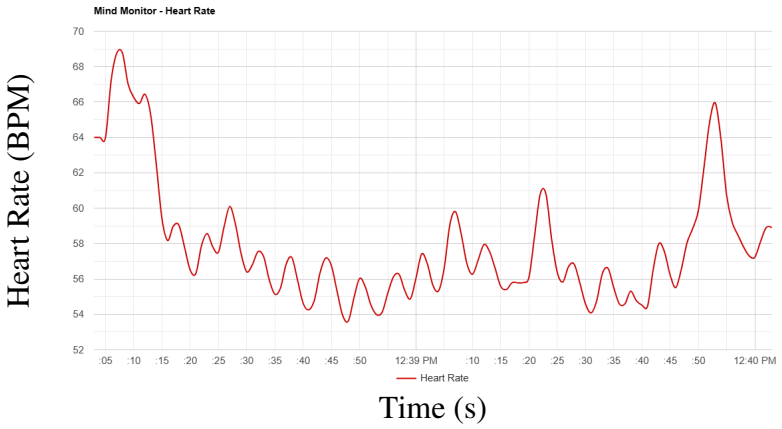


Figure 4.6: P2's heart rate, although more sporadic, doesn't see any major changes over time

range for gamma is between 40Hz and 80Hz, and here we see one participant, P2, with a sub-40Hz frequency. Below 40Hz in gamma is often found in people with more learning disabilities or memory issues. Ultimately, one 60-second extract is not a strong measure of a person's cognitive ability, however, there is already a range of cognitive capability across people starting to be suggested between these two participants.

Another interesting wave from this section of the study was the delta waves. The delta wave is the lowest frequency wave and is a sign of deep sleep and significant restfulness. P1 doesn't have any change here, however, P2 has a much stronger reaction. P2 does noticeably end up normalising around 62.5Hz area, which is very close to where P1 stayed through their song. It is hard to say if this is a coincidence, but there could be some feasibility to the idea that P2 was "woken" up more into a state of cognitive presence while listening to the song. If so, this could act as a better baseline tool for creating a generalised AI model.

A quick look at the respective heart rates of P1 and P2 below in 4.11 and 4.12 shows that both participants had a decrease in their heart rates, however, the data doesn't seem to have normalised around a confident average and thus, although it appears they both relaxed more through this biomarker, a longer recording time would be beneficial in supporting this information. Additionally, there is also the question of does the music genre affect this more than the emotions of the song.

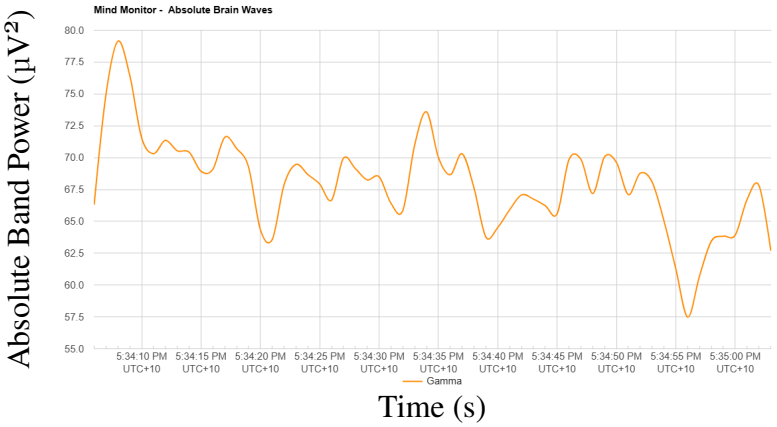


Figure 4.7: P1's gamma wave has a slight average decrease during positive music

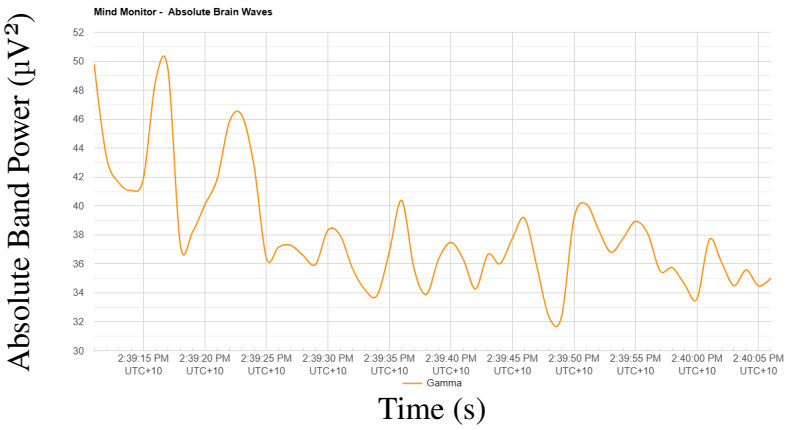


Figure 4.8: P2's gamma wave has a much more significant decrease in reaction to positive music

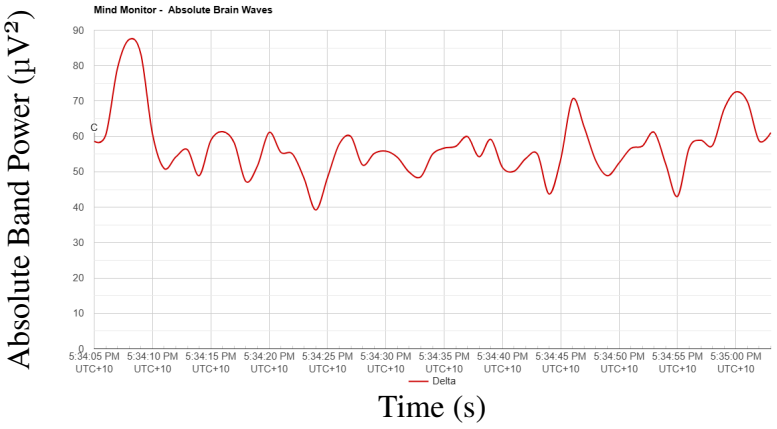


Figure 4.9: P1's delta wave stays very steady through the song

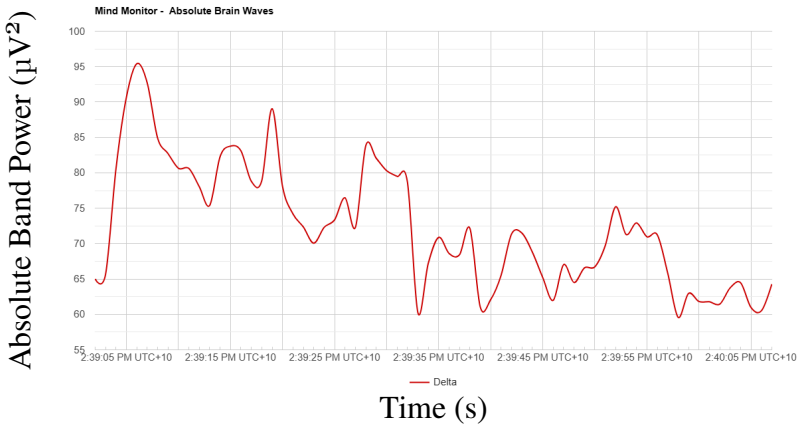


Figure 4.10: P2's delta wave has a much more pronounced decrease over the period

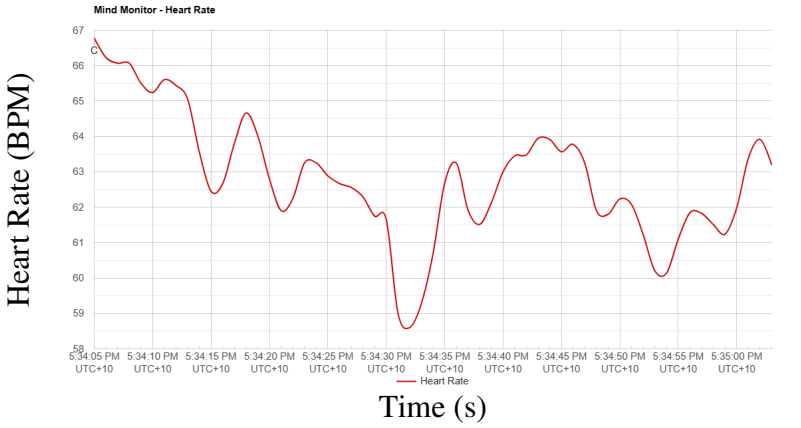


Figure 4.11: P1's heart rate while listening to positive music has a slight decrease

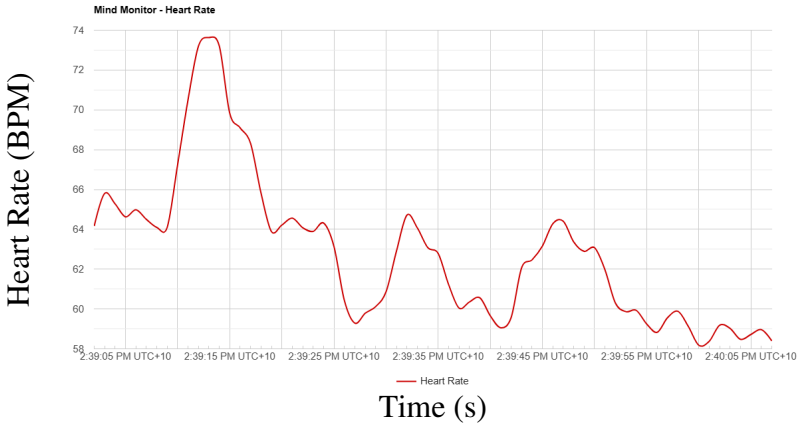


Figure 4.12: P2's heart rate has a slight decrease listening to positive music.

Second Round Of Relaxing Music

More relaxing music was played to attempt to reset the baseline and prevent emotional carryover. P2 interestingly saw no major changes from their brain state in their first relaxation round of music, which is positive for the idea of setting a baseline. P1 did have some interesting results; each brain wave band had normalised around an average slightly lower than their averages in the first round of relaxation music.

Table 4.1: A table showcasing how P1's overall averages of each frequency band decreased on their second round of listening to relaxing music.

Frequency Band	Average in first relaxation period (μV^2)	Average in second relaxation period (μV^2
Delta (δ)	59.89	56.4
Theta (θ)	65.01	64.4
Alpha (α)	89	84.81
Beta (β)	79.66	76.83
Gamma (γ)	66.97	63.9

Negative Valence Music Reaction

Finally, music with a negative valence was played, which is where most of the frequency bands had activity. Both participants' delta, alpha, and gamma waves had some meaningful response to the stimuli.

First, observing the delta frequency band from both P1, figure (4.13) and P2, figure (4.14), we see a contrast between the steady and constant rhythm of P1's waves compared to P2's waves, which are more sporadic and follow a descending pattern. Although P2's waves do seem to start to rise again before the cut-off, leading to the question of what would occur if the recording were longer. Regardless, it is interesting to see the delta waves reacting quite a sporadic amount in P2, which suggests the sad song leads to more active thought and wakefulness since sad songs are often slow or numbing.

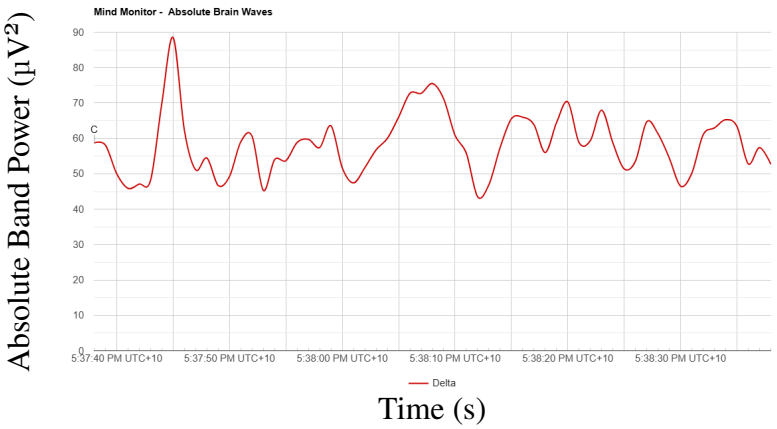


Figure 4.13: P1's delta waves remain very steady throughout the entire recording

The alpha waves are often used to measure one's calmness while they are awake, as well as their alertness. Below in figure 4.15, P1 seems to get more alert and calm as they listen to their negative song, their wave slowly makes gradual increases over the time, before

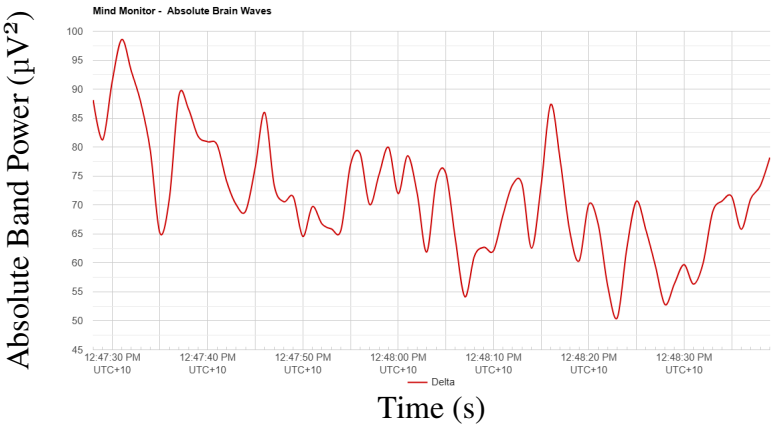


Figure 4.14: P2's delta waves are much more active and dynamic than P1's.

seemingly starting a descent. In contrast, P2's waves are once again more sporadic with lower wavelengths and much much higher amplitudes, this does suggest some instability in their emotion; however, the average of the wave does not change much as it progresses. A complete opposite to P1. There is also another clear challenge of cross-subject variability visualised here, the nature of the alpha waves in figure 4.16 and figure4.15 is very different. It's hard to know for certain if the more sporadic nature of P2 is related to the music or other neurological conditions, such as ADHD, or perhaps it is related to the age of the participant being younger. It is a good reminder that people's brains process stimuli very differently, and the spectrum of neurological differences between people further adds to the unknown in the topic of cross-subject variability.

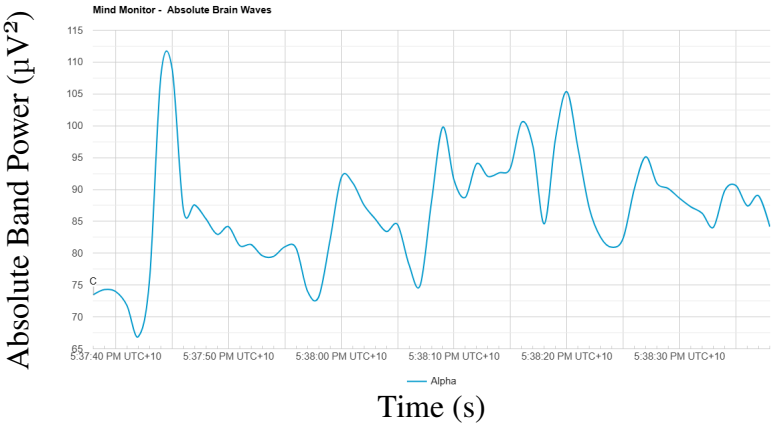


Figure 4.15: P1's alpha waves make a fairly even pattern of accession with a lower amplitude and longer wavelengths

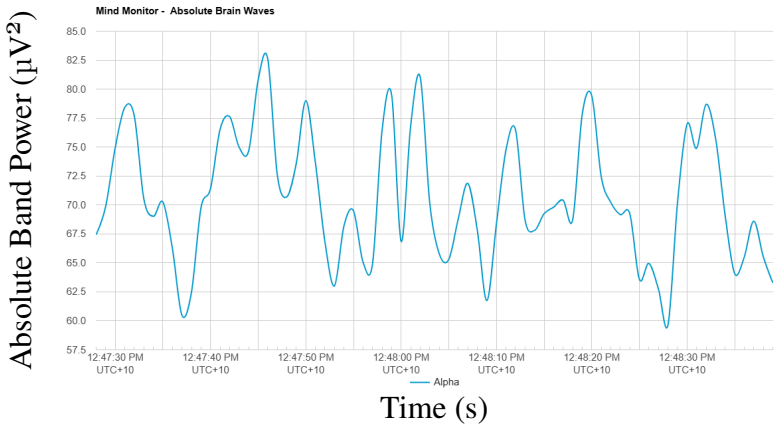


Figure 4.16: P2's alpha waves are again sporadic and uneven, however, they do hover around a consistent 70Hz

Discussion

This experiment was interesting, as it shows that both participants could have strong reactions to stimuli; however, when those reactions occurred (which phase of the experiment they were in), how they occurred (wavelength and amplitude changes, or sometimes rises and decreases in wave frequency), and which frequency band was affected, could often differ between the two participants. Both participants were quite different in age, P1 being around 60 and P2 being around 25, which could affect how these waves act, potentially strongly in how P2's waves often acted more sporadic with higher amplitudes and lower wavelengths. One benefit of having these two participants being far apart in age is that it highlights how cross-subject variability can manifest and pose an issue to EEG AI models. These brains both seem to act quite differently to the human eye, however, the hope is that there will be patterns across these bands that machine learning can pick up that can find similarities. The crucial part of this paper, however, is to introduce methods of reducing this cross-subject variability and the experiment design to have "relaxation" periods to prevent emotional carryover and create a baseline for people taking the experiment. These preliminary results showcase that between two people, there are already noticeable differences in how the two brains react to the stimuli, not only in the trends of the waves but also their structure(their wavelength and amplitude range). This does suggest that creating the baseline across multiple people will be a difficult challenge and perhaps not a feasible solution, however, there is the hope that the models used will be able to detect patterns that a glance at a graph does not show. There is still feasibility in preventing emotional crossover to allow the data from the participants to be more reproducible and consistent; the more opportunities to remove external impacts on the EEG data, the more we can look to mitigate cross-subject variability.

4.3 Visualising Data Artifacts

Another experiment I did to familiarise myself with EEG data was to have a separate participant (P3) listen to music with their eyes closed as per the methodology, and compare it to them listening to music with their eyes open and allowing blinks. The two images can be seen below:

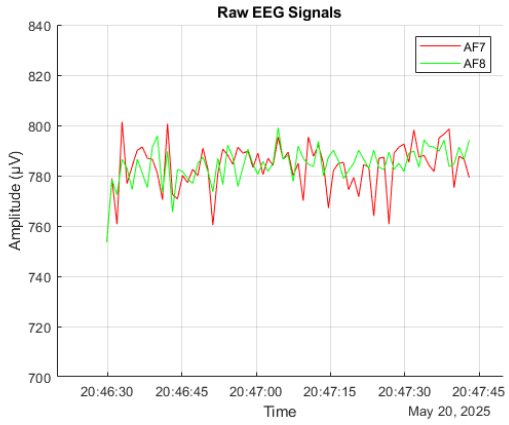


Figure 4.17: Raw EEG Data with eyes closed, no sudden amplitude changes signifies that there are no artifacts

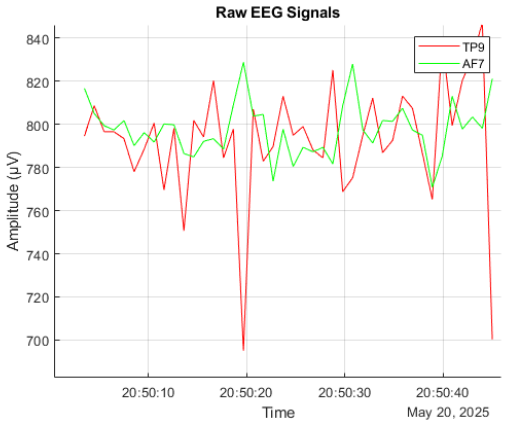


Figure 4.18: Raw EEG Data with eyes open evident with the large spikes where P3 blinked or moved their eyes

Above, in figure 4.17 the red and green data, AF7 and AF8 nodes are fairly straight with little deviation, in contrast, in figure 4.18, AF7 and AF8 have a lot more noise and appear more rugged. Recalling figure 3.1, AF7 and AF8 are the left and right frontal electrodes, it confirms that more activity in the eyes would impact these electrodes. When we blink or move our eyes, we cause very minor shifts in the electric fields around the front of the forehead and the artifacts that we see in 4.18, the more sporadic AF7 and AF8 readings, become much more prevalent. Although there are methods to reduce and remove these artifacts, the best prevention tool is educating subjects beforehand to keep their eyes closed and still and simply focus on the music playing.

5 Results & Discussion

Each architecture was built in a way that allows the program to replicate the same pipeline process for results comparison. Identical processing of the data is paramount to meaningful comparison of both program performance metrics and model performance metrics.

5.1 Machine Specifications

The following results were all collected on the same machine whilst performing as few background tasks as possible to prevent noise from the system. The machine has the following relevant specifications:

Table 5.1: System specifications used during architectural benchmarking.

CPU	AMD Ryzen 5 5600X 6-core Processor
RAM (Total, Speed)	16GB, 3600MHz
OS	Windows 10 Version 22H2
Python Version	3.11.9

5.2 Architecture Performance

Across the architectures, three primary metrics were tracked and measured for a comparison of performance. Firstly, the duration the program ran for was measured. This covers from program start time to end time and essentially measures how efficient the program is overall. The second key metric was how much memory was used in the program; this is the maximum amount of memory the program utilised during its runtime, providing insight into how resourceful the architectures are. Lastly, how well the program utilised the CPU. This represents the proportion of available CPU resources used by the program during its runtime, between a range of 0 and 1, where a lower number means the program placed less load on the processor.

Table 5.2: Models and metrics recorded during benchmarking.

MODEL	ELAPSED TIME (s)	MEMORY USAGE (MB)	CPU UTILISATION (%)
Layered	928.9995	2119.4080	0.1268
Pipes and Filters	900.7490	2263.1863	0.1241
Microkernel	4069.9069	4748.5605	0.1245
Prototype Notebook	842.3496	2872.3763	0.1300

All architectures implemented single-thread programming, with none of the architectures implementing multi-threading. This provides a clear baseline for each architecture's computational overheads without any further optimisation being implemented.

Each Architecture was run 20 times with the following results documenting the average of the multiple runs for better accuracy and measurement of consistency.

5.2.1 Runtime Duration

Time is often a high-priority metric, especially in EEG analysis, where a driving goal in academia is live inference on a subject. Live inference requires minimal delay between the signals entering the system and the emotional label being assigned accurately.

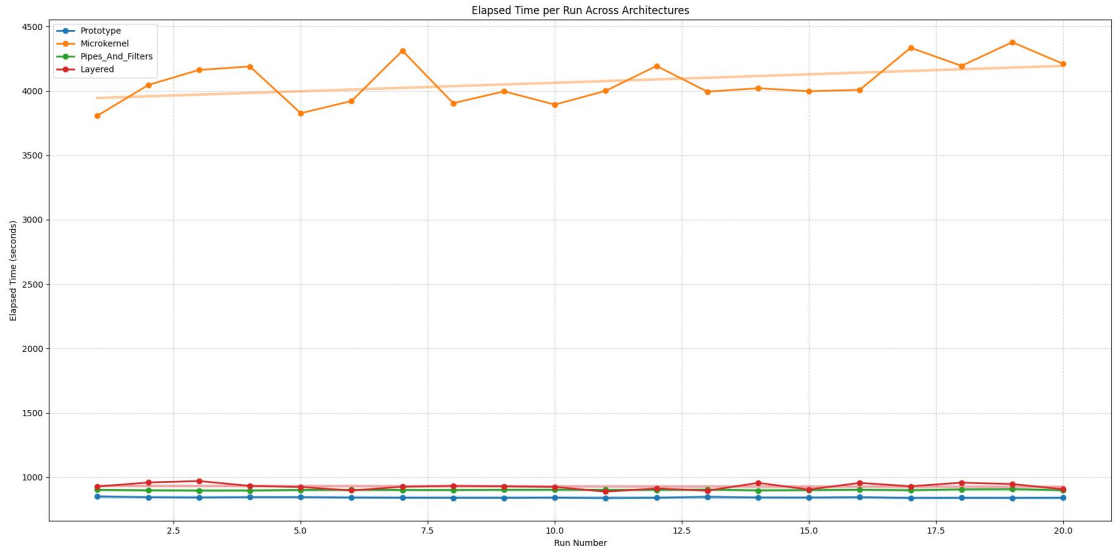


Figure 5.1: A line graph depicting the duration of a full runtime of the program.The Microkernel is significantly more inefficient compared to other architectures.

When comparing all the models as seen in figure 5.1, there is a significant performance difference when comparing Microkernel to the other three architectures. Although projected to be a slower architecture, the microkernel program averages a 338% slower speed than the next slowest architecture. This essentially invalidates the microkernel as a worthwhile investment for any program where runtime is an Architecturally Significant Requirement (ASR). The likely cause of inefficiency here are the communication overheads found in the architecture regarding the data flow needing to be passed through the kernel frequently; although these overheads were believed to impact results, there are hybrid microkernel architectures that could potentially mitigate the primary cause of the slow processing.

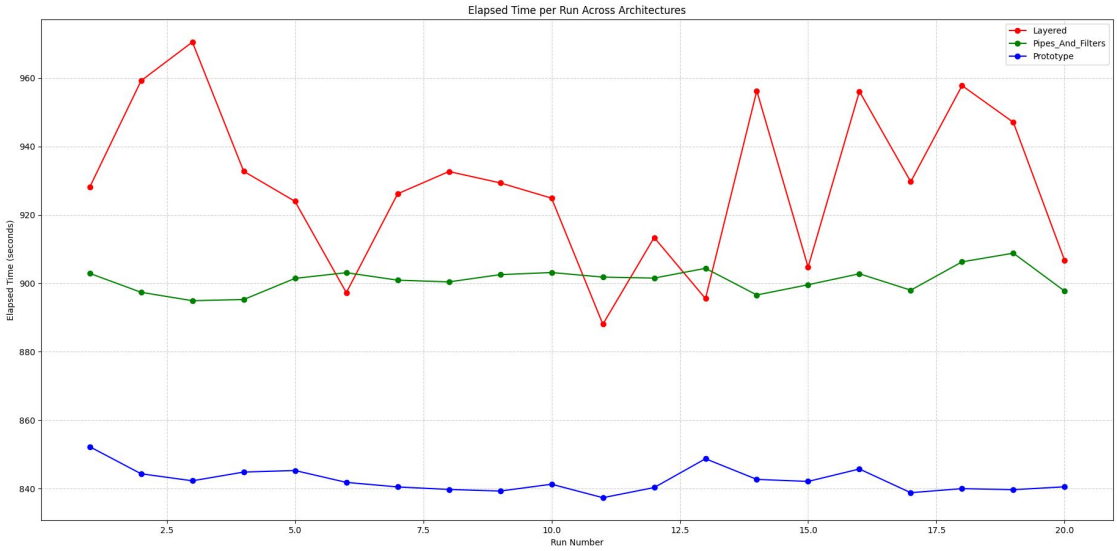


Figure 5.2: A line graph depicting the duration of a full runtime of the program. Microkernel has been removed for better visualisation of the other architectures.

When looking at the other architectures' performances, isolated from the microkernels' data, there are some notable patterns that emerge through the 20 runs.

Ultimately, the fastest architecture was the original prototype Jupyter notebook. An average run took around 14 minutes, and impressively had little spread across instances, with only a 1.8% spread in variation. Approximately 6.9% slower is the Pipes and Filters architecture. Performing the best of the three specifically designed architectures. The architecture also has a small spread of 1.6%, showing it ran very consistently. Pipes and Filters was not faster than the original, likely because the data flow of the prototype behaved as a pseudo-Pipes and Filters architecture, with the filters being Jupyter cells and running linearly through the program. The very similar spread between the two architectures does promote this idea. However, the ultimate benefits of having a structured Pipes and Filters architecture, although slower, are significant, especially in larger-scale projects.

Adding in the comparison of the Layered architecture further supports Pipes and Filters as the most functional choice regarding time-to-run. The Layered architecture averages 3.1% slower than Pipes and Filters; however, it has a comparatively significantly larger spread from its average runtime, with a standard deviation of 8.9% from its mean.

Both the Layered and Pipes and Filters architectures have feasibility when considering runtime performance additional measures could be placed on these architectures to further improve performance, but as a baseline, the architectures both perform soundly. Pipes and Filters does seem to be more consistent with its runtime by having a much smaller standard deviation spread around its average. A Microkernel does not yield initial promises to performance in this metric; however, similar to the other architectures, the baseline may be pushed with modifications to the architecture's structure.

5.2.2 Memory Usage

Memory usage measures the maximum amount of memory used by the program at any point throughout its runtime. A larger amount of memory used implies the programs inefficiency

in cleaning its environment. When working with machine learning, large datasets will always be a key part, and as such, how a program performs in managing and clearing memory is paramount to avoid scalability issues as projects increase in size.

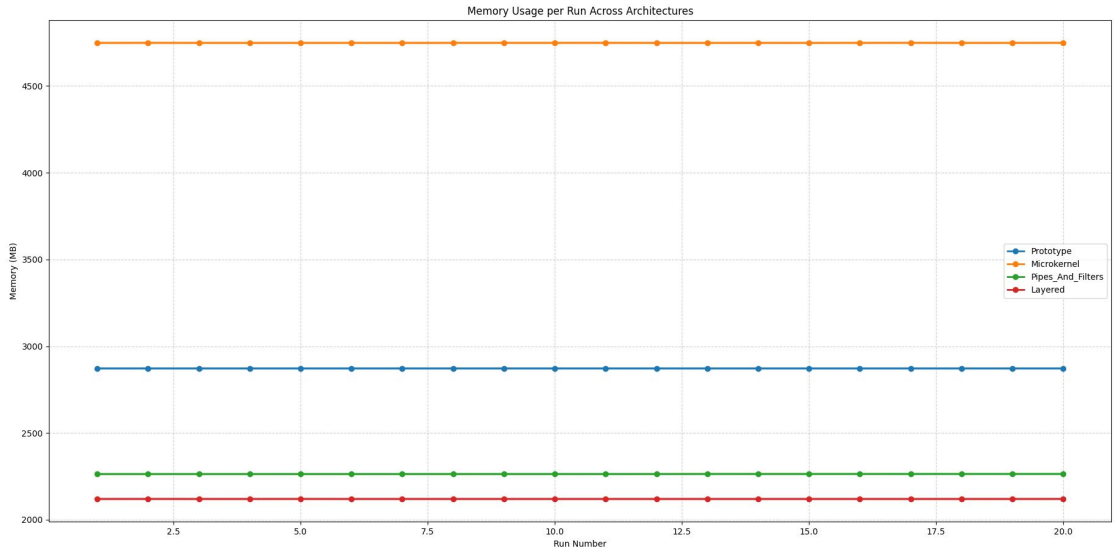


Figure 5.3: The line graph shows that the programs were consistent in their memory habits, and present a clear hierarchy of performance.

In figure 5.3, a clear overview of how each model handles memory can be seen. Once again the worst performing architecture was Microkernel, averaging the use of approximately 4.7GB in its execution. This is the only architecture to perform worse when compared to the initial prototype without following a strict plan of structure. The Prototype acts as a good baseline for technical performance in metrics, and the Microkernel architecture uses up to 65% more memory.

Where the Prototype does perform worse, however, is when compared to the Layered and Pipes and Filters architecture. As previously stated, the Prototype is most similar to the Pipes and Filters, and although the more formal architecture failed to perform better in runtime, it does perform 21% better in its management of memory. Similarly, the Layered architecture also performs well against the Prototype with a 26% increase in performance in this metric. This makes the Layered architecture the best performing in this area, albeit with only a marginal improvement over Pipes and Filters.

5.2.3 CPU Utilisation

The final metric measured was CPU utilisation and represents how much of the available CPU resource the program uses during its runtime. A number closer to 0 represents using fewer resources, whilst a number closer to 1 represents the program needing to use more CPU resources to perform the same task.

Figure 5.4 shows how well each program uses the available computational resources. Ultimately, every architecture performs well in this metric, performing better than the baseline whilst also staying competitive with each other.

Once again, there is similar behaviour between Pipes and Filters and the Prototype, both having least amount of spread from the average while having some minor spikes across their

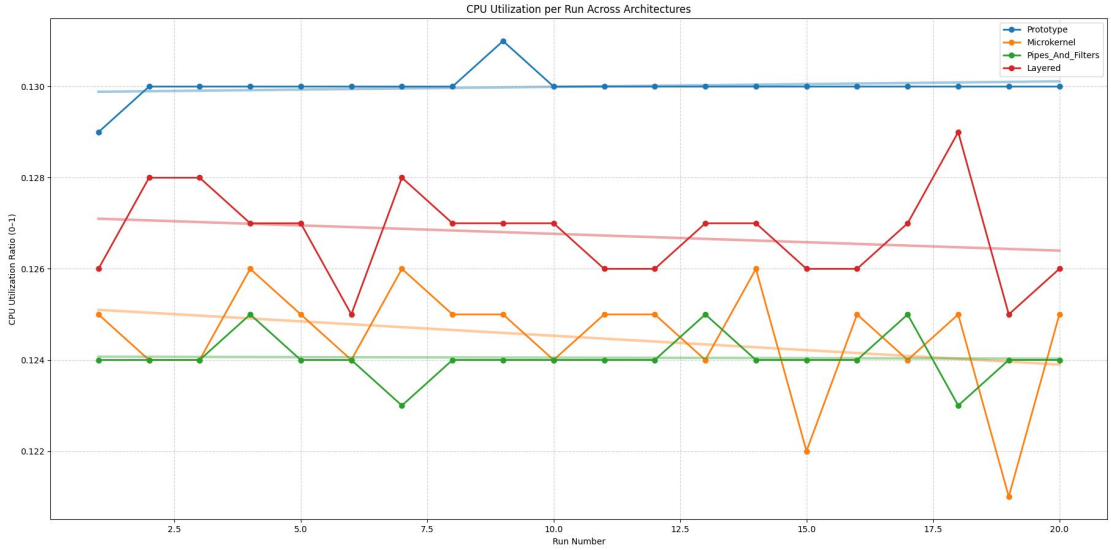


Figure 5.4: A line graph depicting the CPU utilisation metric. A smaller number represents not requiring as much CPU usage to perform tasks.

runs. Conversely, both the Microkernel and Layered architectures have more variation in their CPU utilisation. The Microkernel architecture has a 4% standard deviation, whilst the Layered architecture has a 3% standard deviation.

All the architectures performed similarly in this area, with the means of the three formal architectures all within 0.003 units of each other.

5.3 Model Performance

Four different models performances where tested across the architectures, with multiple configurations being tested. These four algorithms where Support Vector Machine (SVM), Random Forest (RF), K-nearest neighbour(Knn), and a Multi-layer Perceptron classifier (MLP). Each of these algorithms have hyper parameters that can be adjusted to change model performance.

5.3.1 Architectures Relationship to Model Performance

As each architecture includes the same model logic and code wherever possible, as well as handling random seeds, in theory each model should perform identically regardless of which architecture the model code is in. However, this was not the case.

In figure 5.5, it is clear that there are some minor changes in how the models all perform across the architectures. The expected result would have each model have identical bar graphs. The Layered and Prototype do have very close model performance, interestingly, though there are still slight variations. On some occasions, Pipes and Filters is also quite similar to the aforementioned. Looking at the SVM and MLP F1-scores, the Pipes and Filters appear to perform very similarly; however, the recall and precision graphs show there are some differences.

When looking at more specific results across the model configurations, there is now no benefit to looking at each architecture independently. If each architecture's models

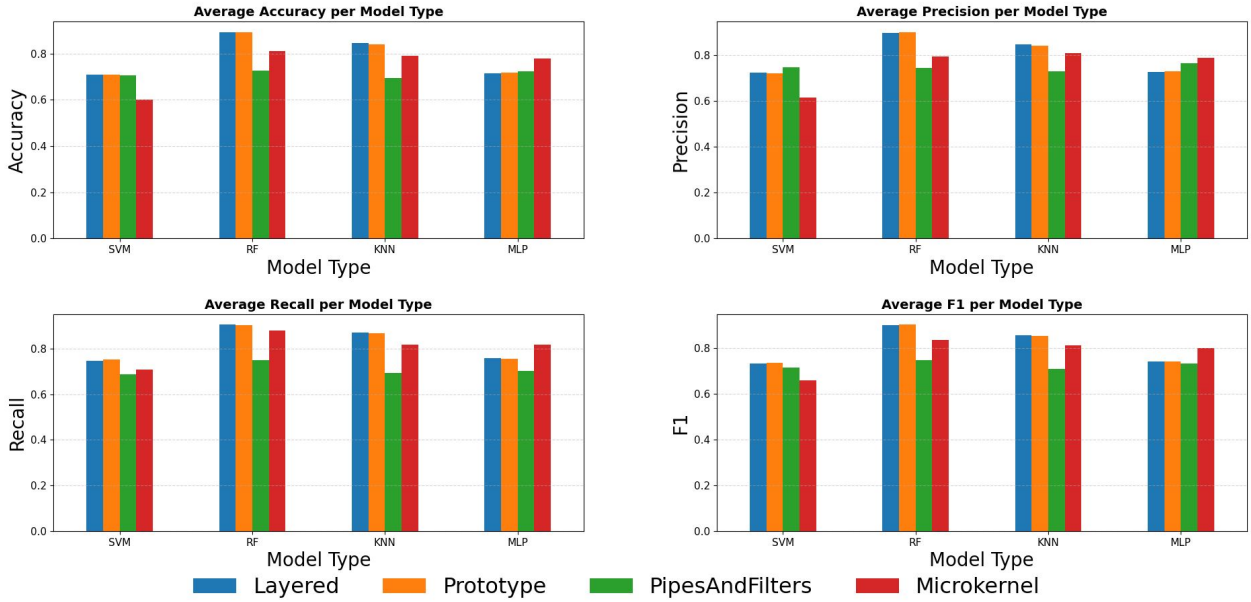


Figure 5.5: A bar graph depicting how different models performed differently with their F1-scores across the architectures.

performed as expected and were identical, analysis would be done specifically on the models' configurations; however, this leads to the architecture being an important factor and as such, below are the tables of each architecture's specific model configurations averaged across their 20 runs. Highlighted are the best-performing configurations per architecture.

Table 5.3: Averages of models performances in the **Layered Architecture**.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
SVM (no LDA, C=0.1)	0.700	0.701	0.765	0.732
SVM (no LDA, C=10)	0.709	0.720	0.747	0.728
SVM (with LDA, C=0.1)	0.716	0.735	0.733	0.734
SVM (with LDA, C=10)	0.716	0.735	0.733	0.734
KNN (k=5)	0.571	0.851	0.921	0.885
KNN (k=7)	0.871	0.851	0.921	0.885
KNN (k=25)	0.792	0.798	0.819	0.808
Random Forest (500 trees)	**0.895**	**0.897**	**0.907**	**0.902**
Random Forest (100 trees)	0.891	0.895	0.903	0.899
MLP (tanh)	0.717	0.734	0.739	0.736
MLP (Logistic)	0.716	0.717	0.775	0.745
MLP (Prelu)	0.716	0.725	0.758	0.741

Table 5.4: Averages of models performances in the **Pipes and Filters Architecture**.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
SVM (no LDA, C=0.1)	0.677	0.702	0.695	0.698
SVM (no LDA, C=10)	0.680	0.724	0.656	0.688
SVM (with LDA, C=0.1)	0.731	0.780	0.701	0.738
SVM (with LDA, C=10)	0.733	0.780	0.701	0.738
KNN (k=5)	0.712	0.746	0.705	0.725
KNN (k=7)	0.703	0.738	0.695	0.716
KNN (k=25)	0.671	0.703	0.675	0.689
Random Forest (500 trees)	0.729	0.744	0.698	0.736
Random Forest (100 trees)	**0.722**	**0.743**	**0.740**	**0.741**
MLP (tanh)	0.720	0.759	0.705	0.731
MLP (Logistic)	0.727	0.771	0.701	0.735
MLP (Prelu)	0.722	0.763	0.701	0.731

Table 5.5: Averages of models performances in the **Microkernel Architecture**.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
SVM (no LDA, C=0.1)	0.596	0.606	0.732	0.663
SVM (no LDA, C=10)	0.597	0.309	0.718	0.659
SVM (with LDA, C=0.1)	0.604	0.621	0.693	0.655
SVM (with LDA, C=10)	0.604	0.621	0.693	0.655
KNN (k=5)	**0.844**	**0.866**	**0.842**	**0.854**
KNN (k=7)	0.813	0.834	0.819	0.826
KNN (k=25)	0.718	0.720	0.787	0.752
Random Forest (500 trees)	0.817	0.8+9	0.888	0.841
Random Forest (100 trees)	0.805	0.791	0.870	0.828
MLP (tanh)	0.795	0.810	0.813	0.811
MLP (Logistic)	0.757	0.758	0.815	0.784
MLP (Prelu)	0.788	0.795	0.821	0.807

Table 5.6: Averages of models performances in the **Prototype Architecture**.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
SVM (no LDA, C=0.1)	0.694	0.700	0.757	0.727
SVM (no LDA, C=10)	0.701	0.707	0.756	0.730
SVM (with LDA, C=0.1)	0.720	0.734	0.750	0.742
SVM (with LDA, C=10)	0.720	0.734	0.750	0.742
KNN (k=5)	0.868	0.852	0.911	0.880
KNN (k=7)	0.864	0.875	0.870	0.872
KNN (k=25)	0.789	0.796	0.816	0.805
Random Forest (500 trees)	**0.900**	**0.907**	**0.907**	**0.907**
Random Forest (100 trees)	0.889	0.894	0.899	0.896
MLP (tanh)	0.716	0.742	0.719	0.730
MLP (Logistic)	0.718	0.713	0.796	0.752
MLP (Prelu)	0.719	0.731	0.751	0.741

Unexpectedly, no one algorithm proved more consistent across the architectures then the others, and, there is little consistency across high performing models in other architectures. Table 5.7 shows that the Knn configuration using k=5 performed fairly strongly through all the models, whereas the Random Forest with 500 trees achieved the best results in any one particular model instance, achieveing 0.902 and 0.907 in the Layered and Prototype architectures respectively. Interestingly however, this configuration dropped drastically to 0.736 in the Pipes and Filters architecture, although this was still the best model performance in the Pipes and Filters architecture.

The consistently worst performing model was the SVM without also using a Linear Discriminant Analysis (LDA), these configurations consistency performed poorly across all the architectures. When adding on the LDA process, the SVM's gained some performance boost, but nothing noticeable.

Table 5.7: Averages of models F1-Scores across all architectures.

MODEL	LAYERED	MICROKERNEL	PIPES AND FILTERS	PROTOTYPE
SVM (no LDA, C=0.1)	0.732	0.663	0.698	0.727
SVM (no LDA, C=10)	0.728	0.659	0.688	0.730
SVM (with LDA, C=0.1)	0.734	0.655	0.738	0.742
SVM (with LDA, C=10)	0.734	0.655	0.738	0.742
KNN (k=5)	0.885	0.854	0.725	0.880
KNN (k=7)	0.885	0.826	0.716	0.872
KNN (k=25)	0.808	0.752	0.689	0.805
Random Forest (500 trees)	0.902	0.841	0.736	0.907
Random Forest (100 trees)	0.899	0.828	0.741	0.896
MLP (tanh)	0.736	0.811	0.731	0.730
MLP (Logistic)	0.745	0.784	0.735	0.752
MLP (Prelu)	0.741	0.807	0.731	0.741

5.4 Live Inference

Live inference was tested as per the methodology. The Python script developed did successfully work, as when the participant who trained the dataset re-listened to the songs used in its creation, which they had labelled with their own bias, the script accurately predicted the correct valence output. The predictions were not immediate, and some windows were predicted inaccurately. The script was labelled as a majority success, as it correctly labelled the bulk of the song's windows. This led to one important observation of live inference of EEG audio recordings; when we average out which label was most commonly predicted in a song, we can find the type of emotion that song elicits. However, some sections of songs with an overall negative valence are still labelled as positive in certain segmented windows. This means that when we create a model and a dataset where the model trains on windows of a song, and the dataset is built with only a single label per song, we are sure to be creating, to some degree, an inaccurate model.

The script was also run with the participant listening to songs not in the training data, where accurate predictions were once again made. Importantly, when talking about accurate predictions, these are based on the participants' self-assessment of how they felt during the song, meaning there is a caveat of self-assessment possibly being unreliable. This project was not designed to test the effectiveness or measure any particular performance metrics; the

primary goal with live inference was to test its plausibility of inferring on a person's emotional state, which this script successfully achieved. Future work could look at developing more accurate connections between these predictions and the neurological state of the participant, and drawing more academically sound results to see if you can find tangible metrics of accuracy in this area.

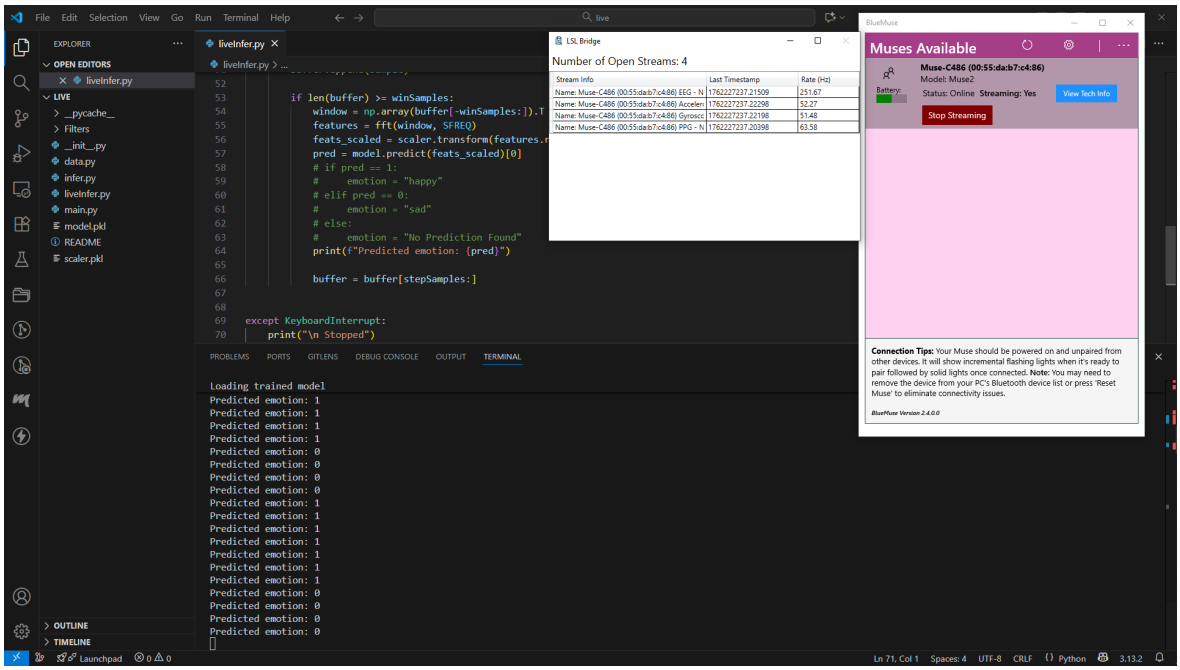


Figure 5.6: A demonstration of live inference working and predicting a valence score of 0 or 1. When a window is inferred on, the prediction is printed to the console.

6 Conclusion & Future Work

6.1 Conclusion

Ultimately, across the architectures, the Microkernel proved the most inefficient and positioned itself as the worst option of the three. Although there are ways to optimise and create hybrid versions of each of these architectures that may boost performance, the Microkernel as a baseline is still significantly poorer than the competition.

Between the Layered and Pipes and Filters architectures, technically, they both perform soundly. The layered architecture does have more sporadic results; however, it trends similarly to the Pipes and Filters, which is very consistent in its performance. Pragmatically, the Pipes and Filters architecture was smoother to implement and was better at matching the ASR's of the project. It was easier for testing and scaling, and was significantly more modular than the Layered approach.

Regarding machine learning models, the results were unexpected. No single model performed significantly above the others consistently, although the Random forest with 500 trees did achieve substantially good results, reaching above 90% twice. As the program is using EEG data across a dataset of people, the performance of these models is skewed higher due to the dataset being full of the same subjects.

Live inference can be done when precautions are made to prevent cross-subject and cross-device variability. When focusing on one participant using one EEG collection device, a dataset can be trained and then used to infer on that person's EEG recordings in real-time. The effectiveness and accuracy of this process should be refined, with the likely expansion of the program to be trained on a multi-class classification problem, not binary.

6.2 Future Work

This project is multifaceted, and future work could expand on numerous avenues of it:

- Testing of more architectures to determine if other, less popular solutions could prove exceptionally well for EEG pipelines.

- Testing of hybrid models. Larger projects often have multiple systems and problems being solved; looking at how modular architectures can be fitted together to bring us more efficiency in an EEG pipeline could yield beneficial results.
- More complex Deep Learning algorithms can be looked at; there are many papers already looking at these models' performance on the DEAP dataset, however.
- Outlining more tangible performance metrics for live inference testing. How to test accuracy, lag, and other metrics to determine how well the script is performing.

7 Appendix

.1 Github Repository

The source code for all architectures, as well as original prototype notebooks and the live inference script, all used in this thesis, can be found in the following GitHub repository:

`https://github.com/KayCeeGit/EEG-Pipeline`

This repository was added to this thesis on 4/11/2025.

.2 Custom Song Dataset

Song and Artist	Valence Score
Apocalypse (Cigarettes after sex)	0
Violin Sonata No.5 in F minor, BWV 1018: I. Largo (Johann Sebastian Bach)	0
Somnus (Yoko Shimomura)	0
From the dining table (Harry Styles)	0
Roaring Tides (Shinji Orito)	0
Trois Pieces Breves I. Allergro (Jacques Ibert)	1
Symphony No.5 in B flat D485, 1st Movement (Franz Schubert)	1
Paprika (Japanese Breakfast)	1
Empyrean (Meganeko)	1
Pyrite Town (Tsukasa Tawada)	1

Table 1: List of songs used for the custom dataset used in live inference.

A
Ethics

References

[1] J. Bajwa, U. Munir, A. Nori, and B. Williams. *Artificial intelligence in healthcare: transforming the practice of medicine*. Future healthcare journal **8**(2), e188 (2021).

[2] *Ieee recommended practice for architectural description for software-intensive systems*. IEEE Std 1471-2000 pp. 1–30 (2000).

[3] R. Cerón, J. Arciniegas, J. Ruiz, J. Dueñas, J. Bermejo, and R. Capilla. *Architectural modelling in product family context*. In *1471-2000 - IEEE Recommended Practice for Architectural Description for Software-Intensive Systems*, vol. 3047, pp. 25–42 (2004).

[4] I. Gorton, F. Khomh, V. Lenarduzzi, C. Menghi, D. Roman, I. Weber, A. Liu, R. Kazman, and P. Pelliccione. *Software architectures for ai systems: State of practice and challenges*. In *Software Architecture*, pp. 25–39 (Springer Nature Switzerland, Cham, 2023).

[5] A. Shah and M. Siddiqui. *Software architecture for machine learning- based systems: Issues, challenges, and solutions* (2023).

[6] R. Nazir, A. Bucaioni, and P. Pelliccione. *Architecting ml-enabled systems: Challenges, best practices, and design decisions*. Journal of Systems and Software **207**, 111860 (2024). URL https://www.sciencedirect.com/science/article/pii/S0164121223002558.

[7] S. Amershi, A. Begel, C. Bird, R. Deline, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann. *Software engineering for machine learning: A case study*. pp. 291–300 (2019).

[8] W. Bischofberger, J. Kühl, and S. Löffler. *Sotograph – a pragmatic approach to source code architecture conformance checking*. In *1471-2000 - IEEE Recommended Practice for Architectural Description for Software-Intensive Systems*, vol. 3047, pp. 1–9 (2004).

[9] L. Bass, R. Kazman, and P. Clements. *Software architecture in practice*. SEI series in software engineering (Addison-Wesley, Upper Saddle River, NJ, 2013 - 2013), third edition. ed.

[10] L. Chen, M. Ali Babar, and B. Nuseibeh. *Characterizing architecturally significant requirements*. IEEE Software - SOFTWARE **30**, 38 (2013).

[11] F. Buschmann and F. Buschmann. *Pattern-oriented software architecture volume 1 : a system of patterns*. Wiley Software Patterns Series (John Wiley Sons, Chichester, 2009 - 1996), 1st edition ed.

[12] F. Buschmann, K. Henney, and D. C. Schmidt. *Pattern-Oriented Software Architecture: A Pattern Language for Distributed Computing, 4th Volume* (Wiley, 2007).

[13] J. Savolainen and V. Myllarniemi. *Layered architecture revisited — comparison of research and practice*. In *2009 Joint Working IEEE/IFIP Conference on Software Architecture European Conference on Software Architecture*, pp. 317–320 (2009).

[14] L. Laibinis and E. Troubitsyna. *Fault tolerance in a layered architecture: a general specification pattern in b*. In *Proceedings of the Second International Conference on Software Engineering and Formal Methods, 2004. SEFM 2004.*, pp. 346–355 (2004).

[15] M. Richards. *Software architecture patterns* (O'Reilly Media, Inc., Sebastopol, CA, 2022), second edition. ed.

[16] E. Fernandez and J. Ortega-Arjona. *The secure pipes and filters pattern*. In *2009 20th International Workshop on Database and Expert Systems Application*, pp. 181–185 (IEEE, 2009).

[17] T. Scheibler, D. Roller, and F. Leymann. *From pipes-and-filters to workflows*. In K. Popplewell, J. Harding, R. Poler, and R. Chalmeta, eds., *Enterprise Interoperability IV*, pp. 255–264 (Springer London, London, 2010).

[18] R. Rana and S. Baul. *A survey on microkernel based operating systems and their essential key components*. SSRN Electronic Journal (2023).

[19] V. Dini. *Microkernel-based web architecture: Design implementation considerations* (2025). URL https://arxiv.org/abs/2502.08802.

[20] H. Chen, X. Miao, N. Jia, N. Wang, Y. Li, N. Liu, Y. Liu, F. Wang, Q. Huang, K. Li, H. Yang, H. Wang, J. Yin, Y. Peng, and F. Xu. *Microkernel goes general: Performance and compatibility in the HongMeng production microkernel*. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 465–485 (USENIX Association, Santa Clara, CA, 2024). URL https://www.usenix.org/conference/osdi24/presentation/chen-haibo.

[21] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. *sel4: formal verification of an os kernel*. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pp. 207–220. ACM (ACM, New York, NY, USA, 2009).

[22] R. Sutter, P. W. Kaplan, and D. L. Schomer. *Historical Aspects of Electroencephalography* (Niedermeyer's Electroencephalography: Basic Principles, Clinical Applications, and Related Fields, 7 edn, 2017).

[23] S. Phadikar, N. Sinha1, and R. Ghosh. *A Survey on Feature Extraction Methods for EEG Based Emotion Recognition* (Department of Electrical Engineering, NIT Silchar, Silchar, Assam, India AND Department of Information Technology, Gauhati University, Gauhati, Assam, India, 2020).

[24] W. Li, W. Huan, B. Hou, Y. Tian, Z. Zhang, and A. Song. *Can emotion be transferred?—a review on transfer learning for eeg-based emotion recognition*. IEEE TRANSACTIONS ON COGNITIVE AND DEVELOPMENTAL SYSTEMS, VOL. 14, NO. 3, SEPTEMBER 2022 (2022).

[25] P. A. Abhang, B. W. Gawali, and S. C. Mehrotra. *Chapter 2 - technological basics of eeg recording and operation of apparatus*. In P. A. Abhang, B. W. Gawali, and S. C. Mehrotra, eds., *Introduction to EEG- and Speech-Based Emotion Recognition*, pp. 19–50 (Academic Press, 2016). URL https://www.sciencedirect.com/science/article/pii/B9780128044902000026.

[26] M. Abo-Zahhad, S. Ahmed, and S. N. Seha. *A new eeg acquisition protocol for biometric identification using eye blinking signals*. International Journal of Intelligent Systems and Applications (IJISA) (2015).

[27] M. D. C. Vipin Gupta and R. B. Pachori. *Cross-subject emotion recognition using flexible analytic wavelet transform from eeg signals*. IEEE SENSORS JOURNAL, VOL. 19, NO. 6, MARCH 15, 2019 (2019).

[28] W.-B. Jiang, X.-H. Liu, W.-L. Zheng, and B.-L. Lu. *Seed-vii: A multimodal dataset of six basic emotions with continuous labels for emotion recognition*. IEEE Transactions on Affective Computing 2024 (2024).

[29] S. Chaudhary, S. Taran, V. Bajaj, and S. Siuly. *Flexible analytic wavelet transform based features for physical action identification using semg signals*. Computer Methods and Programs in Biomedicine Volume 187 (2020).

[30] P. Ekman. *Facial expressions of emotion: New findings, new questions*. Psychological Science **3**(1), 34 (1992). URL http://www.jstor.org/stable/40062750.

[31] J. T. Larsen, S. H. Hemenover, N. C. J., and J. T. Cacioppo. *Turning adversity to advantage: on the virtues of the coactivation of positive and negative emotions*. A psychology of human strengths: Fundamental questions and future directions for a positive psychology pp. 211–225 (2003).

[32] C. T. Fong. *The effects of emotional ambivalence on creativity*. The Academy of Management Journal **49**(5), 1016 (2006). URL http://www.jstor.org/stable/20159814.

[33] A. Ben-Zeev. *The nature of emotions*. Philosophical Studies: An International Journal for Philosophy in the Analytic Tradition **52**(3), 393 (1987). URL http://www.jstor.org/stable/4319927.

[34] B. OFOGHI1, M. MANN1, and K. VERSPOOR1. *Towards early discovery of salient health threats:a social media emotion classification technique*. Biocomputing 2016Proceedings of the Pacific Symposium (2016).

[35] B. Ofoghi1 and K. Verspoor. *Textual emotion classification: An interoperability study on cross-genre data sets*. AI 2017: Advances in Artificial Intelligence. 30th Australasian Joint Conference (2017).

[36] J. A. Russell. *How shall an emotion be called?* (American Psychological Association, 1997). URL https://api.semanticscholar.org/CorpusID:172980480.

[37] Y.-S. Seo and J.-H. Huh. *Automatic emotion-based music classification for supporting intelligent iot applications*. Electronics (Basel) **8**(2), 164 (2019).

[38] J. Russell. *A circumplex model of affect*. Journal of Personality and Social Psychology **39**, 1161 (1980).

[39] S. A. Y. Al-Galal, I. F. T. Alshaikhli, A. W. bin Abdul Rahman, and M. A. Dzulkifli. *Eeg-based emotion recognition while listening to quran recitation compared with relaxing music using valence-arousal model*. 2015 4th International Conference on Advanced Computer Science Applications and Technologies (ACSAT) (2015).

[40] H. J. S. h. B. C. S. b. M. C. A. m. Sungjoo Hwang, A.M.ASCE hwangsj@ewha.ac.kr and M. s. A. SangHyun Lee. *Measuring workers' emotional state during construction tasks using wearable eeg*. Journal of Construction Engineering and Management (2018).

[41] H. K. S. Alsayigh and A. S. M. Khidhir. *Deep learning-based mental task classification using a muse 2 eeg headset*. 6TH INTERNATIONAL CONFERENCE ON ENGINEERING SCIENCES – ICES2022 (2024).

[42] W.-L. Zheng and B.-L. Lu. *Investigating critical frequency bands and channels for eeg-based emotion recognition with deep neural networks*. IEEE Trans. Auton. Mental Develop., vol. 7, no. 3 (2021).

[43] H.-R. Hou, X.-N. Zhang, and Q.-H. Meng. *Odor-induced emotion recognition based on average frequency band division of eeg signals*. Journal of Neuroscience Methods (2020).

[44] M. Asghar, M. Khan, H. Shahid, N. Shorfuzzaman, M.AND Xiong, and R. Mehmood. *Semi-skipping layered gated unit and efficient network: Hybrid deep feature selection method for edge computing in eeg-based emotion classification*. IEEE Access (2021).

[45] M. Bouchane, W. Guo, and S. Yang. *Hybrid cnn-gru models for improved eeg motor imagery classification*. Sensors **25**(5) (2025). URL https://www.mdpi.com/1424-8220/25/5/1399.

[46] L. Z. J. Z. Q. X. Shuaiqi Liu, Xu Wang and S.-H. Wang. *Subject-independent emotion recognition of eeg signals based on dynamic empirical convolutional neural network*. IEEE/ACM TRANSACTIONS ON COMPUTATIONAL BIOLOGY AND BIOINFORMATICS, VOL. 18, NO. 5, (2021).

[47] F. Zheng1, B. Hu1, X. Zheng, C. Ji, J. Bian, and X. Yu. *Dynamic differential entropy and brain connectivity features based eeg emotion recognition*. International Journal of Intelegence Systems (2022).

[48] S. Koelstra, C. Muhl, M. Soleymani, J.-S. Lee, A. Yazdani, and T. Ebrahimi. *Deap: A database for emotion analysis ;using physiological signals*. IEEE Transactions on Affective Computing (2011).

[49] Z. Gao, X. Wang, Y. Yang, Y. Li, K. Ma, , and G. Chen. *A channel-fused dense convolutional network for eeg-based emotion recognition*. IEEE TRANSACTIONS ON COGNITIVE AND DEVELOPMENTAL SYSTEMS, VOL. 13, NO. 4, (2021).

[50] W. Liu, W.-L. Zheng, and B.-L. Lu. *Emotion recognition using multimodal deep learning*. International Conference on Neural Information Processing **9948** (2016).

[51] X. Hu, J. Chen, F. Wang, and D. Zhang. *Ten challenges for eeg-based affective computing*. Brain Science Advances **5**(1), 1 (2019). https://doi.org/10.1177/2096595819896200, URL https://doi.org/10.1177/2096595819896200.

[52] J. J. Bird, D. R. Faria, L. J. Manso, A. Ekart, and C. D. Buckingham. *A deep evolutionary approach to bioinspired classifier optimisation for brain-machine interaction*. Complexity (2019).

[53] OpenAI. *Chatgpt* (2025). URL https://openai.com/.

[54] Grammarly. *Grammarly* (2025). URL https://app.grammarly.com/.

[55] E. K. Center. *Primo research assistant* (2025). URL https://multisearch.mq.edu.au/discovery/researchAssistant.